Advanced Computer-Aided VLSI System Design

Chapter 1. Logic Synthesis & Low-Power Design (1)(2)

Lecturer: Chun-Hao Chang

Graduate Institute of Electronics Engineering, National Taiwan University



DCSL



Outline

- Review on Previous Logic Synthesis
- Physical Synthesis
- Low-power Design and UPF Flow
- Conformal Low-power Check

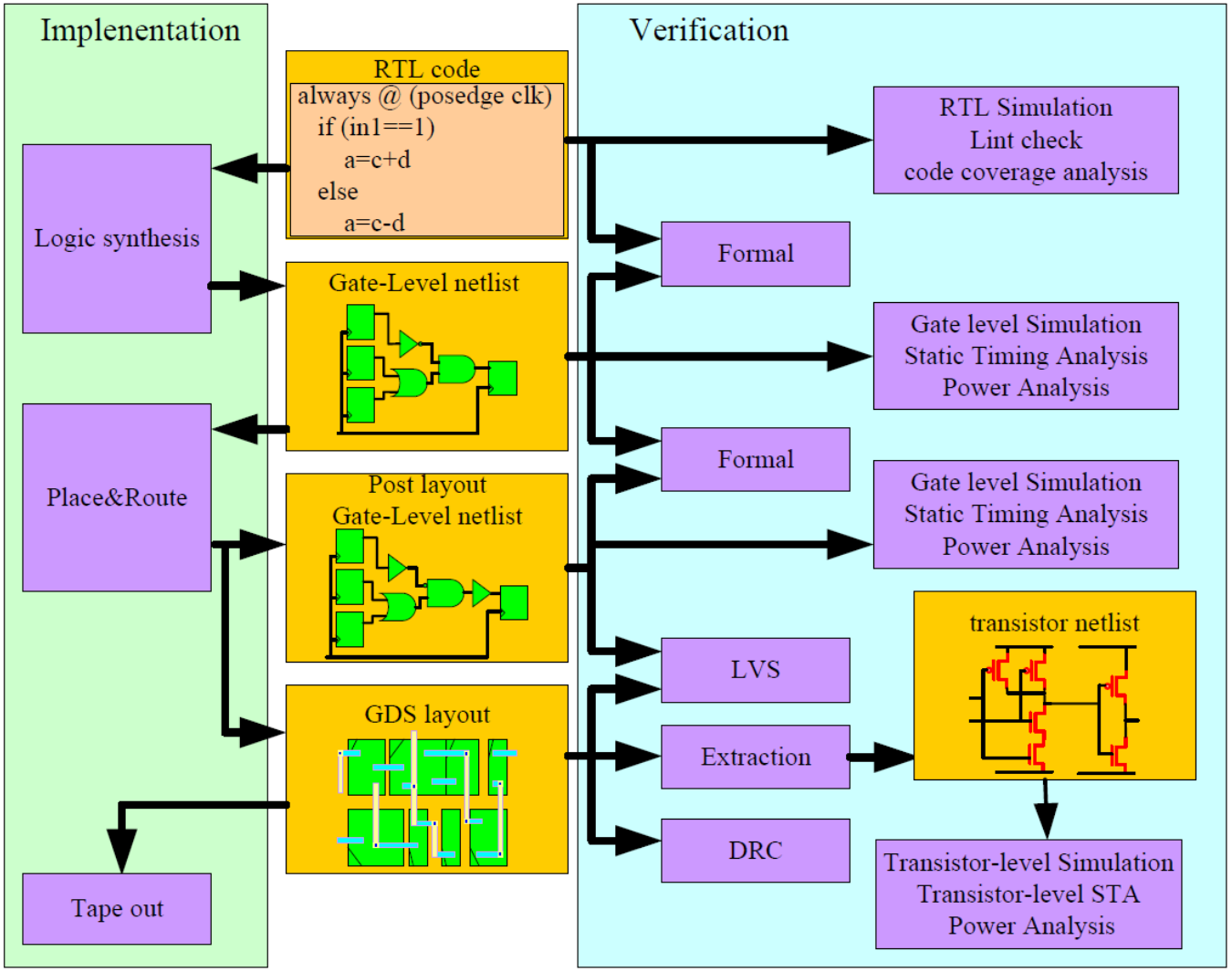


Outline

- **Review on Previous Logic Synthesis**
- Physical Synthesis
- UPF Low-power Design Flow
- Conformal Low-power Check



Cell-based IC Design Flow





Logic Synthesis

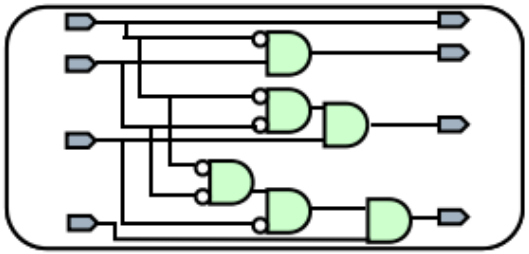
Synthesis = translation + optimization + mapping

```
if(high_bits == 2'b10)begin
  residue = state_table[i];
end
else begin
 residue = 16'h0000;
end
```

HDL Source (RTL)

Translate (HDL Compiler)

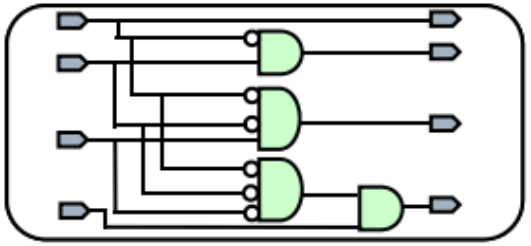
No Timing Info.



Generic Boolean (GTECH)

Optimize + Mapping (Design Compiler)

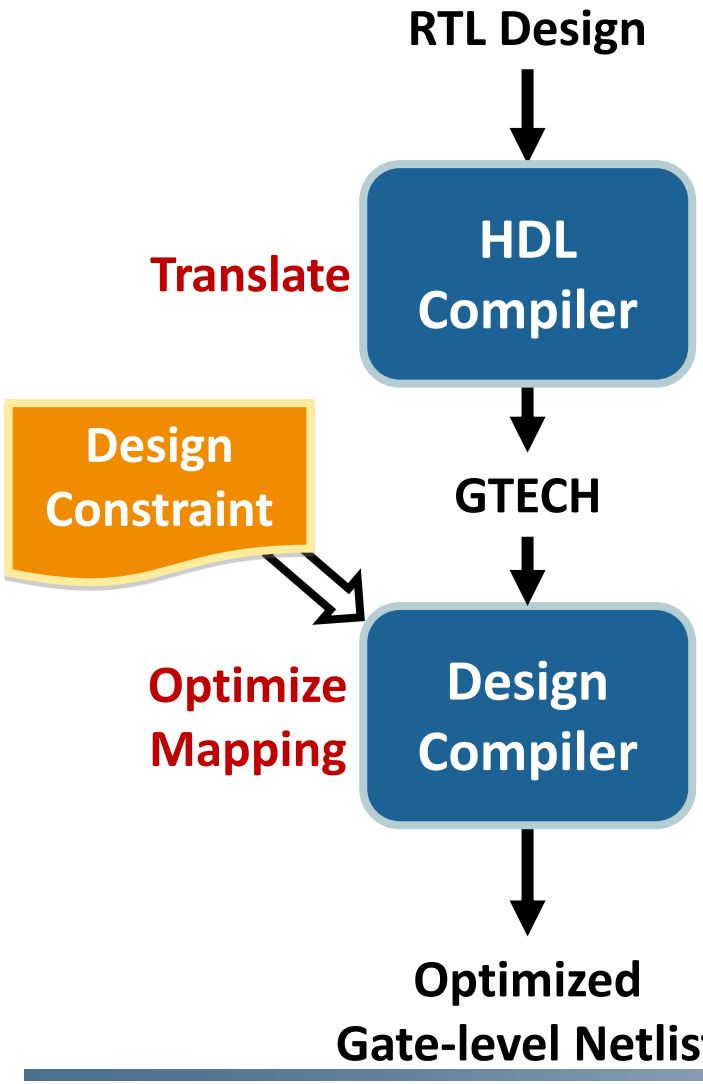
Timing Info. Constraints

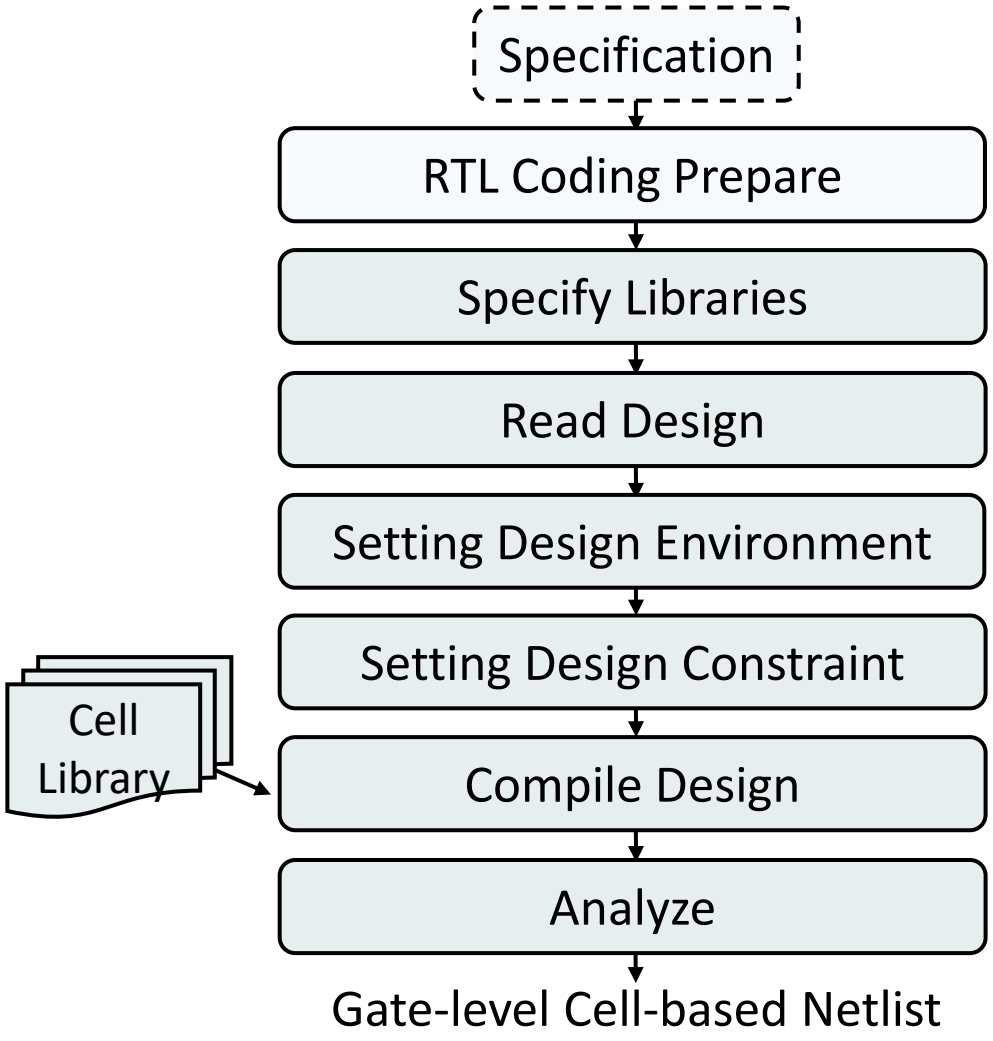


Target Technology



Synthesis Design Flow







Logic Synthesis Example

- A process to convert RTL into a technology-specific gate-level netlist optimized with a set of design constraints
  - Library
    - Standard cell library
  - User data
    - RTL design
    - Design constraints

```
module counter (
    input i_clk, rstn, load,
    input [1:0] in,
    output reg [1:0] out);
    always@(posedge i_clk) begin
        if(!rstn) out <= 2'b0;
        else if (load) out <= in;
        else out <= out+1;
    end
endmodule
```

RTL Design
counter.v



Standard Cell Lib.

Design Constraints

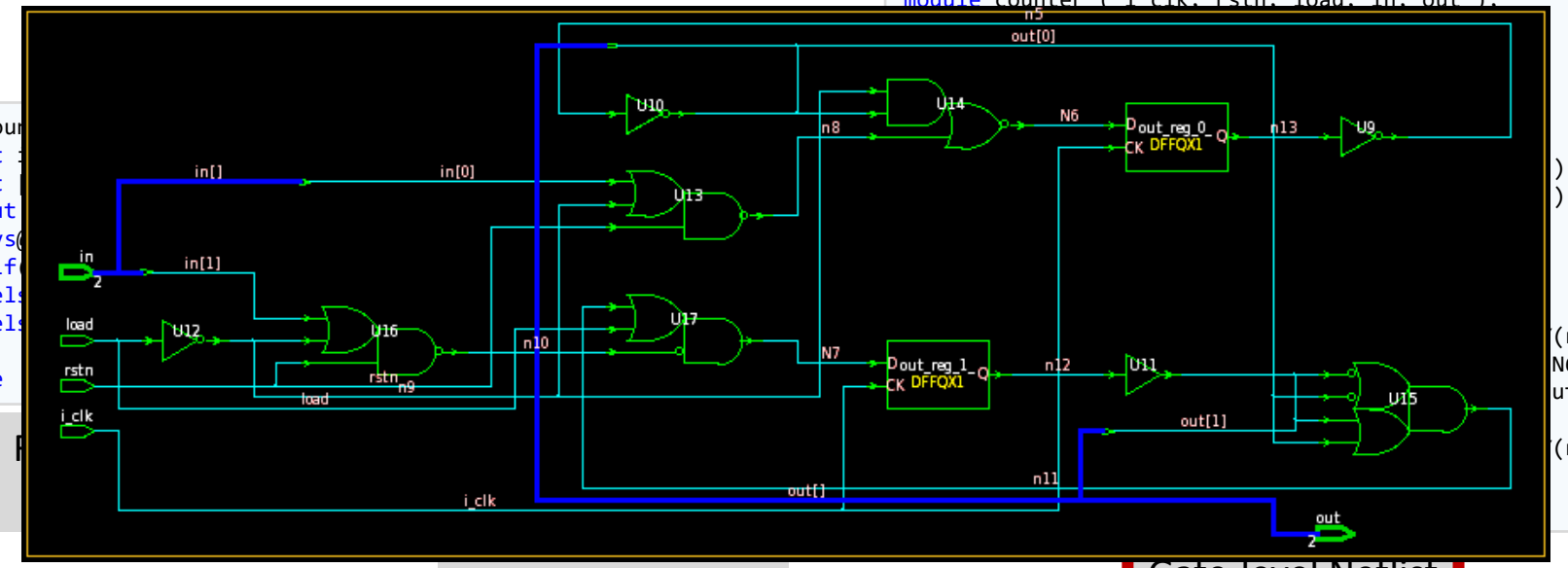
```
module counter ( i_clk, rstn, load, in, out );
  input [1:0] in;
  output [1:0] out;
  input i_clk, rstn, load;
  wire   n12, n13, N6, N7, n5, n8, n9, n10, n11;

  DFFQX1 out_reg_1_ ( .D(N7), .CK(i_clk), .Q(n12) );
  DFFQX1 out_reg_0_ ( .D(N6), .CK(i_clk), .Q(n13) );
  CLKINVX1 U9 ( .A(n13), .Y(n5) );
  INVX16 U10 ( .A(n5), .Y(out[0]) );
  BUFX16 U11 ( .A(n12), .Y(out[1]) );
  INVXL U12 ( .A(load), .Y(n9) );
  OAI21XL U13 ( .A0(in[0]), .A1(n9), .B0(rstn), .Y(n8) );
  AOI21XL U14 ( .A0(n9), .A1(out[0]), .B0(n8), .Y(N6) );
  AOI2BB2X1 U15 ( .B0(out[1]), .B1(out[0]), .A0N(out[1]),
                  .A1N(out[0]), .Y(n11) );
  OAI21XL U16 ( .A0(in[1]), .A1(n9), .B0(rstn), .Y(n10) );
  AOI2BB1X1 U17 ( .A0N(n11), .A1N(load), .B0(n10), .Y(N7) );
endmodule
```

Gate-level Netlist
counter_syn.v

Synthesized Netlist

- Gate-level Netlist
  - Describes the circuit with standard cells



Gate-level Netlist
counter_syn.v



Setting Technology Library

- Technology library includes information of standard cells
  - Operating conditions
    - Process, voltage, temperature
  - Wire load model
  - Standard cell descriptions
    - Functionality, pin info.
    - Timing, area, power info.
  - Etc.

Same set of cells, different PVT conditions

```
fast.db (FF Corner)
typical.db (TT Corner)
slow.db (SS Corner)
```



Technology Library

- Standard cell descriptions
  - Combinational cell & sequential cell list of the technology
  - Detailed **area, power, timing, function** info. of each cell
- Provides basic elements to form gate-level netlist

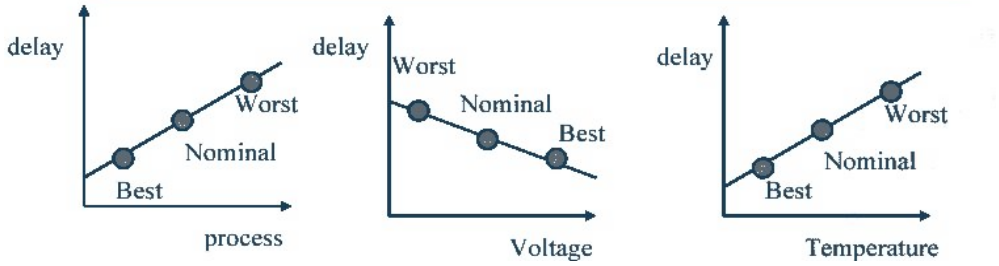
```
cell (NAND2BX1) {
  cell_footprint : nand2b;
  area : 6.789600;
  pin(AN) {
    direction : input;
    capacitance : 0.001359;
  }
  pin(B) {
    direction : input;
    capacitance : 0.002767;
  }
  pin(Y) {
    direction : output;
    capacitance : 0.0;
    function : "(!(!AN B))";
    internal_power() {
      related_pin : "AN";
      rise_power(energy_template_7x7) {
        index_1 ("0.042, 0.066, 0.112, 0.206, 0.392, 0.764, 1.5");
        index_2 ("0.00079, 0.002054, 0.00474, 0.010112, 0.020856, 0.042186, 0.08532");
        values ( \
          "0.003768, 0.003806, 0.003858, 0.003891, 0.003848, 0.003657, 0.003192", \
          "0.003744, 0.003780, 0.003827, 0.003858, 0.003820, 0.003634, 0.003170", \
          "0.003750, 0.003777, 0.003815, 0.003845, 0.003812, 0.003634, 0.003179", \
          "0.003795, 0.003808, 0.003829, 0.003848, 0.003819, 0.003654, 0.003213", \
          "0.003976, 0.003976, 0.003975, 0.003967, 0.003926, 0.003768, 0.003340", \
          "0.004549, 0.004535, 0.004513, 0.004410, 0.004341, 0.004171, 0.003750", \
          "0.005815, 0.005774, 0.005726, 0.005691, 0.005496, 0.005223, 0.004791");
      }
      fall_power(energy_template_7x7) {
        index_1 ("0.042, 0.066, 0.112, 0.206, 0.392, 0.764, 1.5");
        index_2 ("0.00079, 0.002054, 0.00474, 0.010112, 0.020856, 0.042186, 0.08532");
        values ( \
          "0.005118, 0.005214, 0.005344, 0.005457, 0.005528, 0.005567, 0.005591", \
          "0.005100, 0.005197, 0.005326, 0.005439, 0.005511, 0.005551, 0.005576", \
          "0.005113, 0.005210, 0.005342, 0.005459, 0.005535, 0.005578, 0.005606", \
          "0.005164, 0.005254, 0.005380, 0.005493, 0.005572, 0.005620, 0.005652", \
          "0.005377, 0.005454, 0.005565, 0.005669, 0.005743, 0.005795, 0.005832", \
          "0.006123, 0.006079, 0.006074, 0.006168, 0.006224, 0.006269, 0.006305", \
          "0.007591, 0.007527, 0.007456, 0.007403, 0.007346, 0.007374, 0.007400");
      }
    }
    timing() {
      related_pin : "AN";
      timing_sense : positive_unate;
      cell_rise(delay_template_7x7) {
        index_1 ("0.042, 0.066, 0.112, 0.206, 0.392, 0.764, 1.5");
        index_2 ("0.00079, 0.002054, 0.00474, 0.010112, 0.020856, 0.042186, 0.08532");
        values ( \
          "0.103606, 0.115100, 0.137951, 0.181525, 0.266709, 0.434270, 0.772003", \
```



Setting Operating Condition

- Operating condition model scales components delay, directs the optimizer to simulate variations in **process**, **voltage**, and **temperature**.

Name	*Process*	*Temp*	*Volt*	*Interconnection Model*
WCCOM	1.32	100.00	2.7	worst_case_tree
BCCOM	0.73	0.00	3.6	best_case_tree
NCCOM	1.00	25.00	3.3	balance_tree



- General operating conditions in technology process
  - Best case (fast) / Typical case / **Worst case (slow)**
- ***Attributes / Operating Environment / Operating Condition***

```
set_operating_conditions -max_library slow -max slow -min_library fast -min fast
```

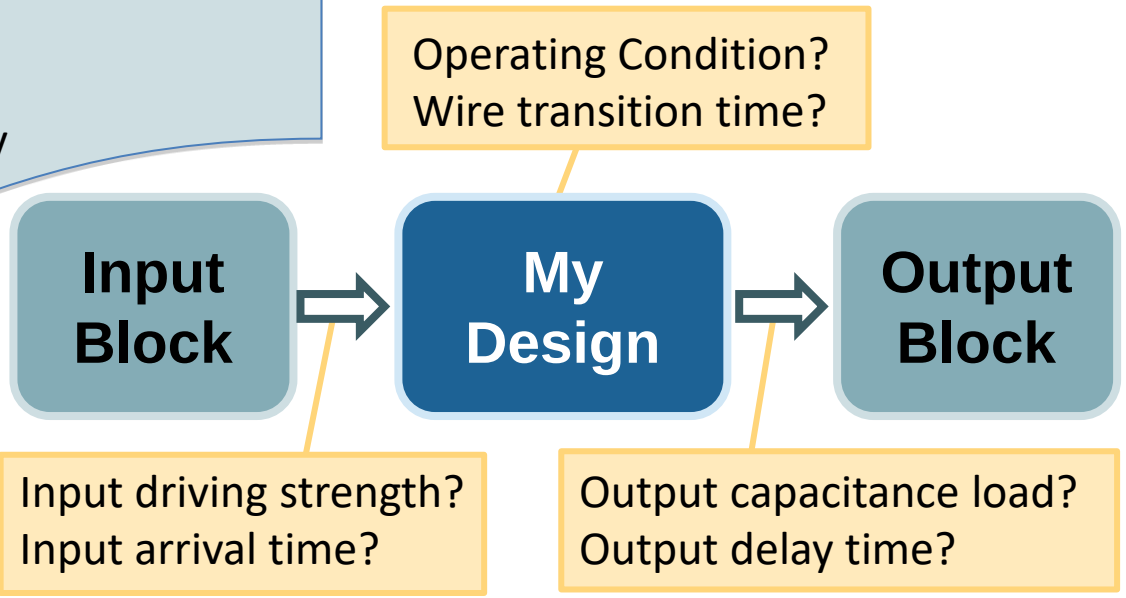


Describing Design Environment - 1

For STA, every node should be known:

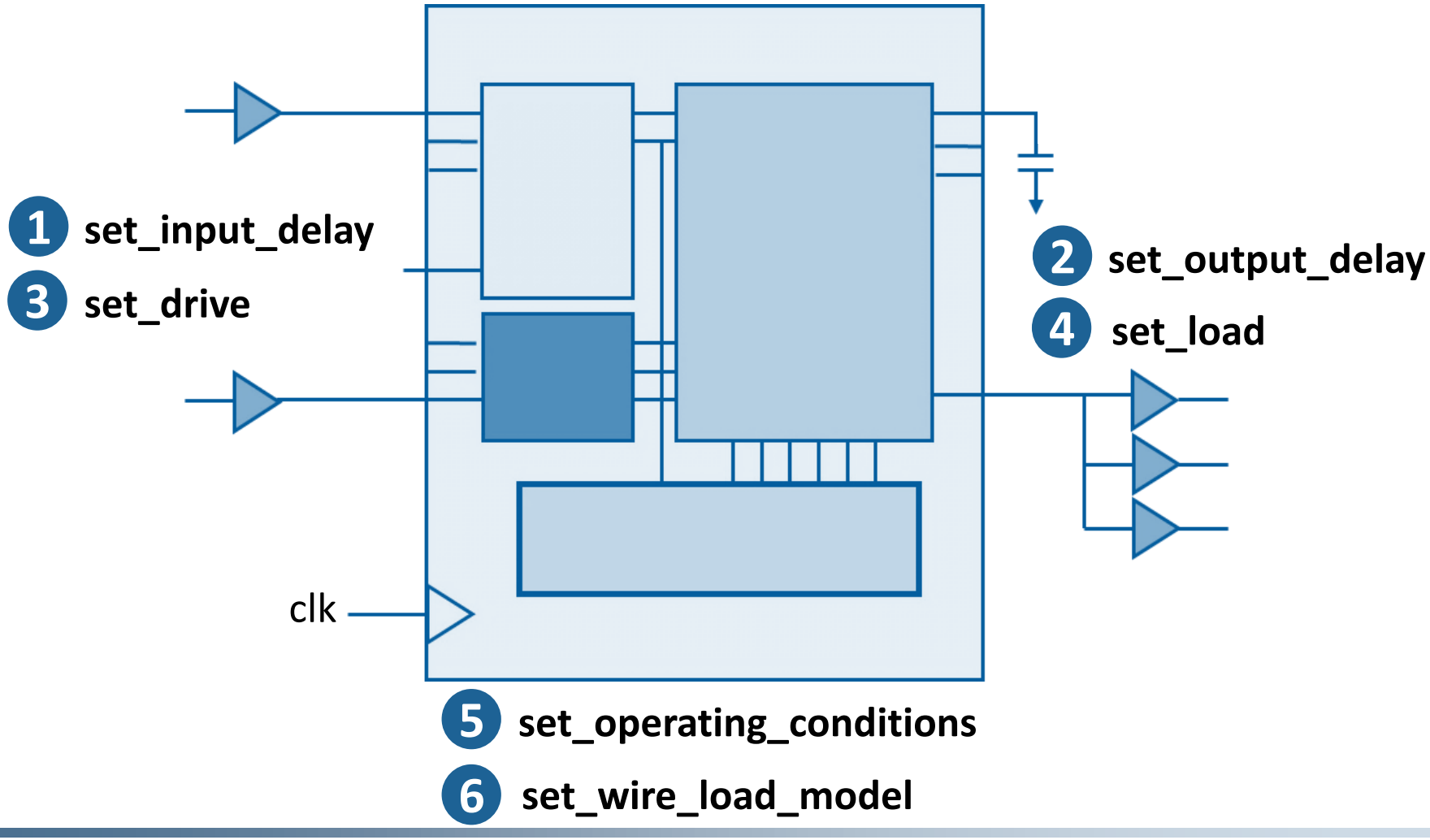
- **RC value**
  - Inside DUT: Tech lib
    1. standard cell/external-IP
    2. wire load model
  - Outside DUT:
    1. set_drive / set_driving_cell
    2. set_load
- **I/O delay**
  - set_input_delay
  - set_output_delay

STA: static timing analysis
DUT: device under test



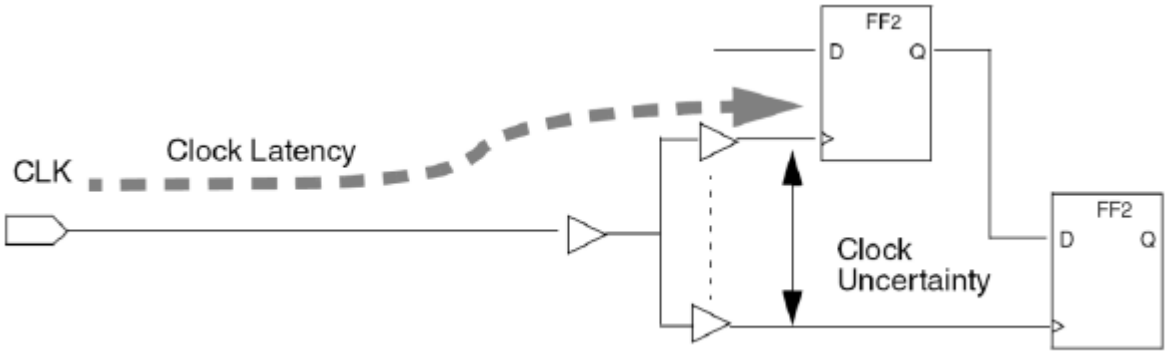


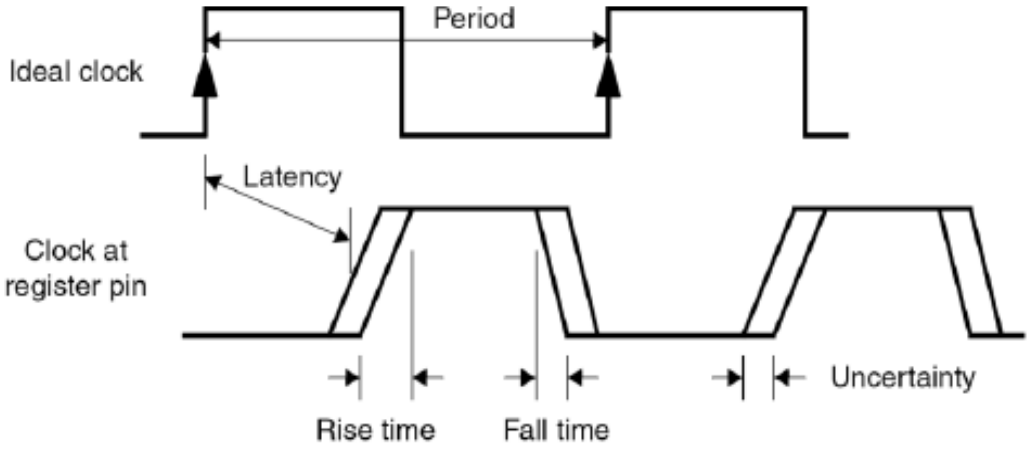
Describing Design Environment - 2



Setting Clock Constraints

- Period
- Waveform
- Uncertainty
  - Skew
  - Jitter
- Latency
  - Source latency (optional)
  - Network latency
- Transition
  - Clock transition
  - Input transition







Setting Clock Uncertainty

- Variation in clock arrival time
  - Models clock skew + jitter effects on the clock
  - Experience
    - Small circuits: 0.1ns
    - Large circuits: 0.3ns

```
set_clock_uncertainty 0.1 [get_clocks clk]
```

```
memory_8k_64_2r_2c/aru/U488/Y (MXI4X1)                    0.40      10.69 r
memory_8k_64_2r_2c/aru/U668/Y (NOR2X1)                    0.09      10.77 f
memory_8k_64_2r_2c/aru/data_in_sc[0] (aru)                0.00      10.77 f
memory_8k_64_2r_2c/sc_memory/D[0] (sc_memory)             0.00      10.77 f
data arrival time                                                   10.77

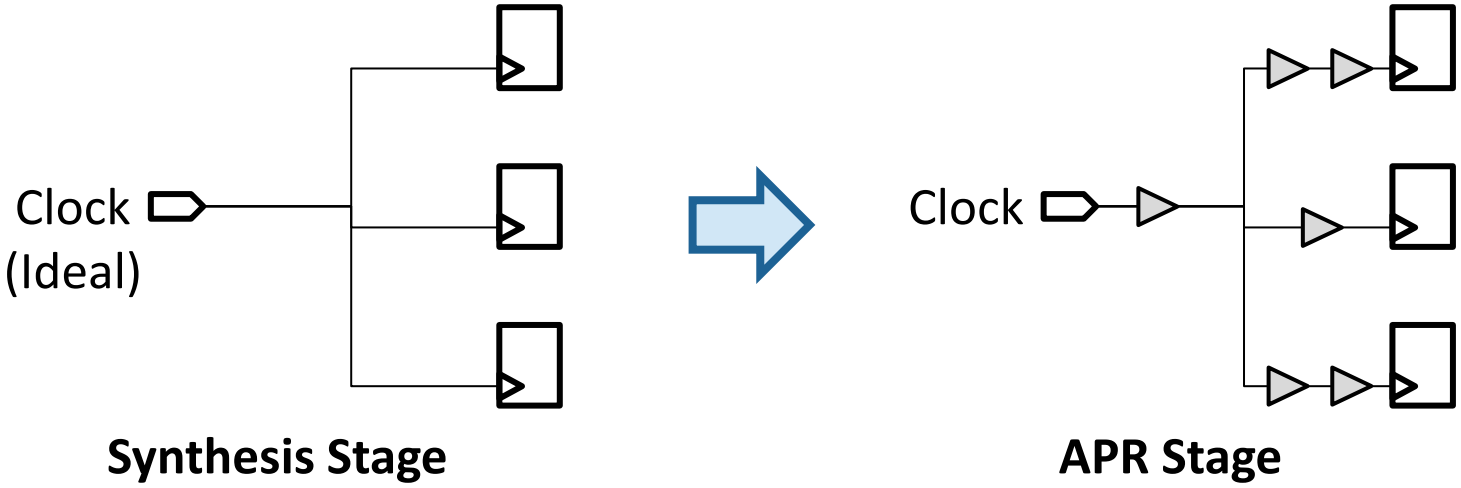
clock clk (rise edge)                                    10.00      10.00
clock network delay (ideal)                               1.00      11.00
clock uncertainty                                        -0.10      10.90
memory_8k_64_2r_2c/sc_memory/CLK (sc_memory)              0.00      10.90 r
library setup time                                       -0.12      10.78
data required time                                                  10.78
-----------------------------------------------------------------------------
data required time                                                  10.78
data arrival time                                                  -10.77
-----------------------------------------------------------------------------
slack (MET)                                                         0.00
```



Clock Modeling in Synthesis & APR

- Clock modeling in automatic place and route (APR) stage
  - Clock tree synthesis is performed
  - A buffer/inverter tree is built for the clock signal

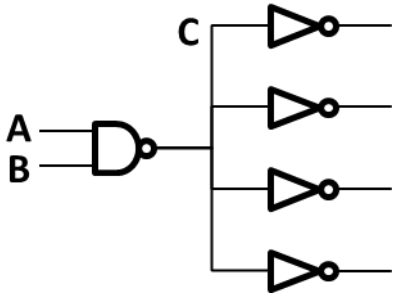
- Clock modeling in synthesis stage
  - Ideal network + clock uncertainty + clock latency





Wire RC Modeling-Wire Load Model

- Fanout of a wire/pin
  - The **number of load gates** connected to the output of the driving gate
- WLM contains information about
  - A lookup table to estimate wire length by its fanout
    - Extrapolation for fanouts not in list
  - Area/capacitance/resistance coefficients
- Note: WLM only provide a rough estimation, area/delay of wire will be more accurate in automatic place and route (APR) stage



Part of gate-level netlist

Fanout	Length
1	4.1
2	5.3
3	6.4
4	7.4

WLM

Area Coeff.	0.7
Cap. Coeff.	1e-4
Resist. Coeff.	8e-8

Fanout of C = 4

⬇

Estimated length = 7.4

⬇

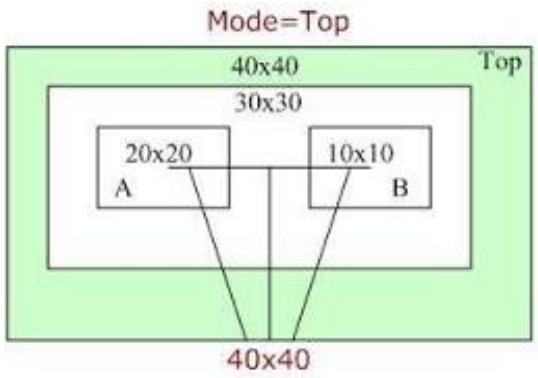
Estimated Area = 7.4*0.7

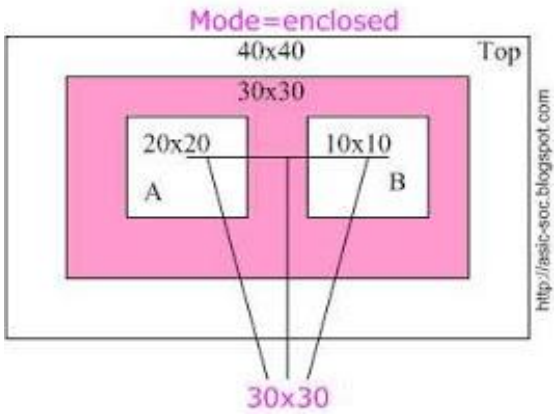
Estimated Cap. = 7.4*1e-4

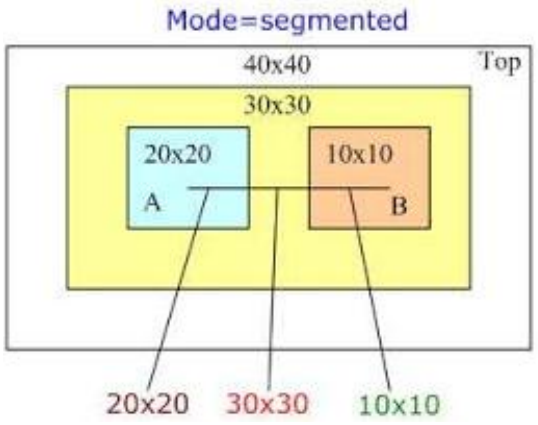
Estimated Resist. = 7.4*8e-8

Wire Load Modes

- 3 hierarchical wire load modes
- Top
  - Applying the wire load models specified for the top level of the design to all nets, the most rough estimation
- Enclosed
  - Using the wire load model which inherits from its upper-level design
- Segmented
  - Nets crossing hierarchical boundaries are divided into segments

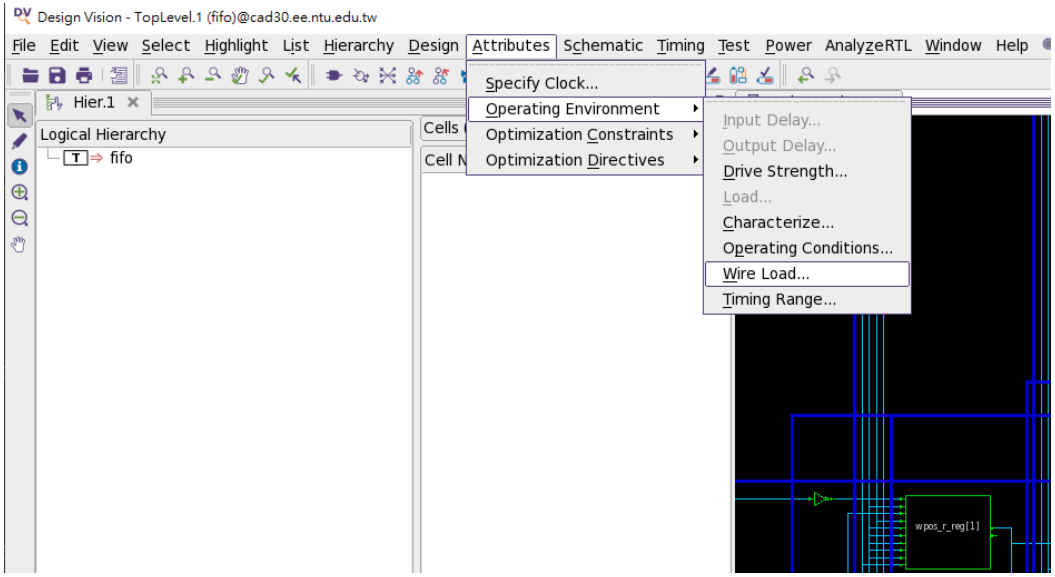


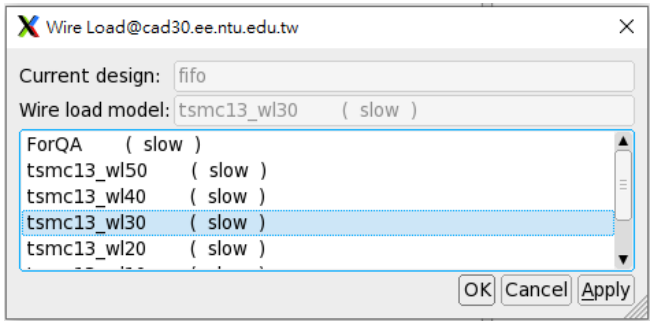




Setting Wire Load Model (WLM)

- ***Attributes/Operating Environment/Wire Load***






```
set_wire_load_model -name tsmc13_w10 -library slow
set_wire_load_mode top
```



Forget About Wire Load Model ☺

"After this lesson, you don't need WLM anymore! And you will have higher Timing Correlation for Syn-to-APR Netlist."



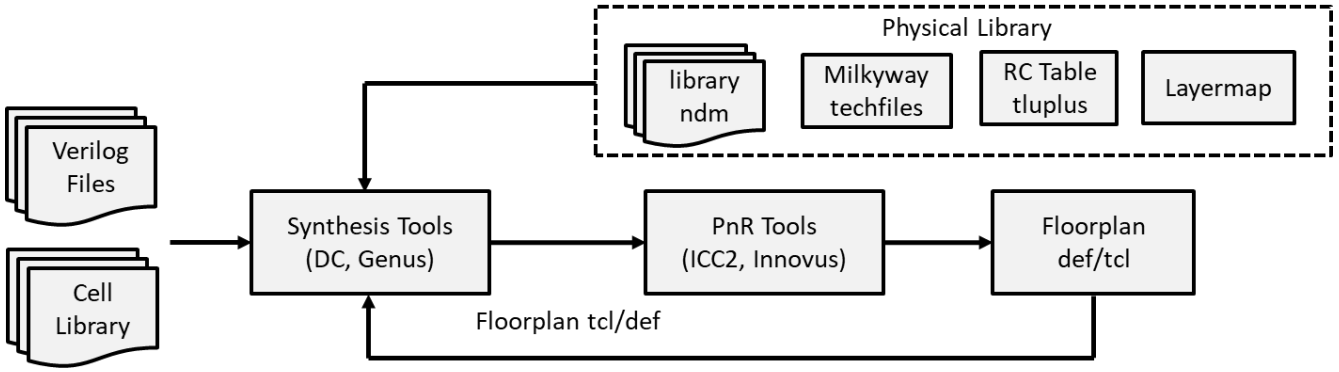
Outline

- Review on Previous Logic Synthesis
- **Physical Synthesis**
- Low-power Design and UPF Flow
- Conformal Low-power Check



What is Physical Synthesis?

- Physical synthesis aims to synthesize RTL file to gate-level netlist with coarse placement
- Why we need physical synthesis?
  - **Wire load model is inaccurate** in advanced node technology (< 40nm)
    - Poor timing correlation with APR
  - Traditional synthesis cannot reflect on global route congestion issue
  - Cell-to-cell distance should be considered





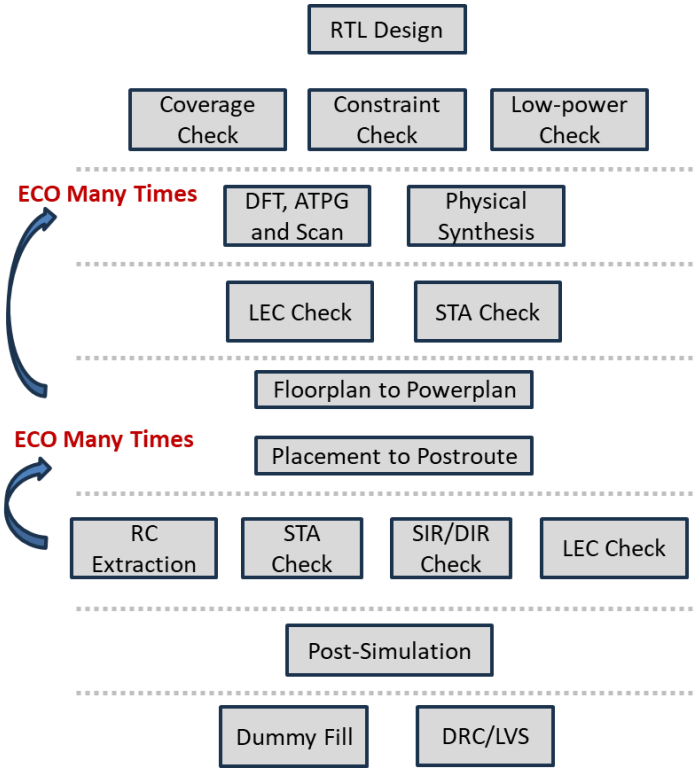
Comparison Table to Different Type of Design Compiler

Technology	Timing Calculation	Activation	Enable Instruction	Additional Feature	Abbreviation
DC Expert	Wire load model	dcnxt	compile	Basic PPA optimization	DC
DC Ultra	Wire load model	dcnxt_shell	compile_ultra	Retime, Re-structure, and Advanced Optimization Algorithm	
	Timing analysis with physical library and floorplan	dcnxt_shell –topo	compile_ultra	Coarse Placement Auto leakage optimization	DC-T
			compile_ultra -spg	Congestion Optimization Multi-corner Muti-mode (UPF)	DC-G



Industrial Golden Flow

- Simulation, waveform and coverage: VCS+Verdi
- Lint and CDC/constraint/low-power: Spyglass
- ATPG and DFT: TestMax-DFT
- Synthesis: DC-G
- STA and timing ECO: PrimeTime (PTPX)
- LEC low-power and formal Check: Conformal-LP (CLP)
- APR floorplan to powerplan (ICC2)
- APR placement to postroute (Innovus)
- RC extraction (ICC2+StarRC, Innovus+QRC, Calibre RCX)
- Post route timing ECO (Tweaker+PTPX)
- Dynamic IR (Ansys Redhawk)
- Post simulation with mixed-signal or analog circuits (VCS+XA, VCS+Finesim)
- DFM and DRC/LVS (Calibre)

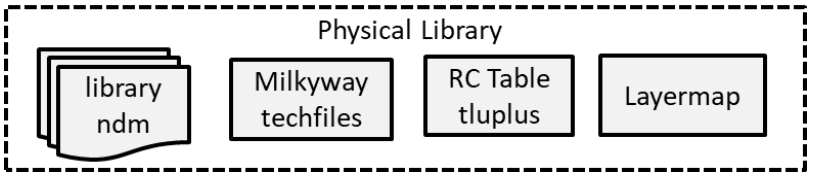




Design Compiler – DC Topological (DC-T)

- DC-T use physical library and floorplan to calculate the path delay
  - Improve path delay correlation with APR tool
  - Reduce unrealistic net delay by wire load model

- Library ndm: stdcell or macro with timing and physical information (LEF or frame)
- Floorplan:
  - .def file: Multi-platform support (Synopsys or Cadence)
  - floorplan.tcl: Only support ICC/ICC2
- Techfiles: milkyway techfile along with layer mapping file (.tf+layermap)
- RC Table: (tluplus file)
- Output: Floorplan tcl or def + netlist



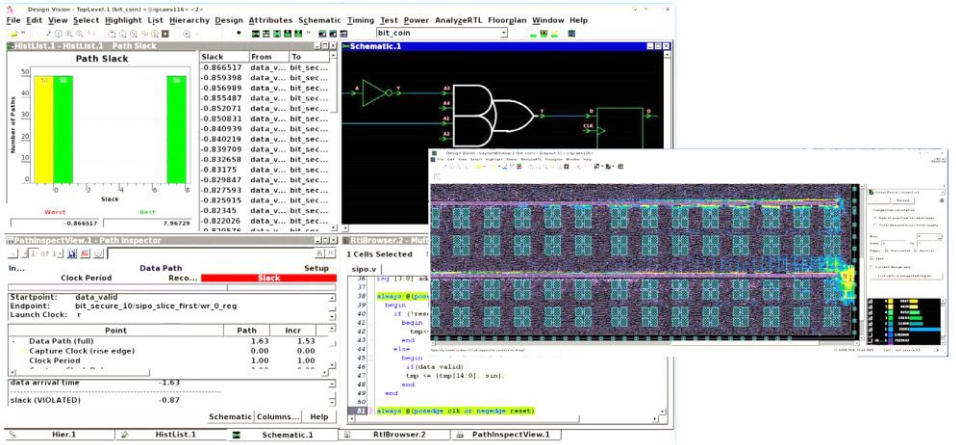


Design Compiler – DC Graphical (DC-G)

- DC-G adds the congestion aware feature by including virtual routing
  - Estimation of net wire length
  - Compute global route congestion
  - Netlist optimization by congestion map
    - Datapath restructuring
    - Reduce pin density by lowering utilization of high fan-in cell

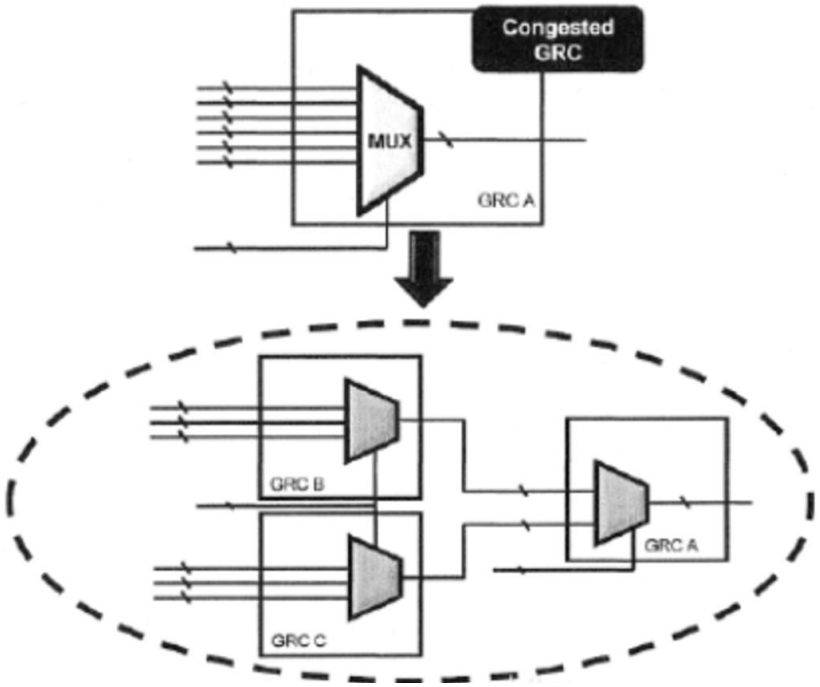
- Activation of DC-G
  - compile_ultra –spg

- Cross-probing with RTL, netlist, and Layout with GUI (**dcnxt_shell -topo -gui**)



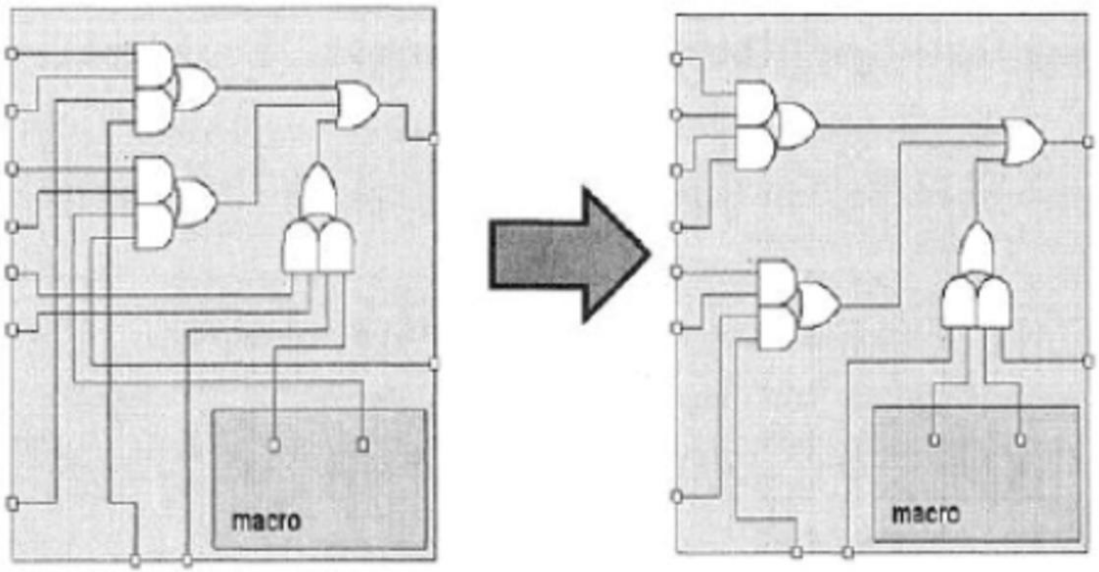
compile_ultra –spg (1)

- MUX restructuring by lowering high fan-in cells usage (lowering pin density)



compile_ultra –spg (2)

- Associative-Commutative Tree Decomposition
  - Improved congestion by decomposing and restructuring the AND/OR Trees based on port or macro pins locations





Physical Synthesis – Environment Setup

- Example: Using ARM 40nm Stdcell

```
set design_lib ${DESIGN}.nlib
set ndm_ref_libs { \
    /DATA/ARCHIVE/techlib/sm40ll/stdcell/be/ndm/sc9mc_logic0040ll_base_rvt_c40.ndm \
    /DATA/ARCHIVE/techlib/sm40ll/stdcell/be/ndm/sc9mc_logic0040ll_base_rvt_c50.ndm \
    /DATA/ARCHIVE/techlib/sm40ll/stdcell/be/ndm/sc9mc_logic0040ll_base_lvt_c40.ndm \
    /DATA/ARCHIVE/techlib/sm40ll/stdcell/be/ndm/sc9mc_logic0040ll_base_lvt_c50.ndm \
    /DATA/ARCHIVE/techlib/sm40ll/stdcell/be/ndm/sc9mc_logic0040ll_base_hvt_c50.ndm \
}

sh rm -rf $design_lib

create_lib -ref_libs $ndm_ref_libs -technology /DATA/ARCHIVE/techlib/sm40ll/tech/milkyway/sc9mc_tech.tf $design_lib
check_library
set_tlu_plus_files -max_tluplus /DATA/ARCHIVE/techlib/sm40ll/tech/milkyway/RCMAX.tluplus -tech2itf_map /DATA/ARCHIVE/techlib/sm40ll/tech/milkyway/tluplus.map
extract_physical_constraints ./pr_floorplan.def
```

```
# Create a design library with reference cell ndm and techfile
create_lib –ref_lib {cell.ndm macro.ndm} \
              –technology xxx.tf \
              your_design.nlib

# Check the library
check_library

# Setup tluplus file for RC delay estimation
set_tlu_plus_files –max_tluplus xxx.tluplus –tech2itf xxx.layermap

# Extract floorplan from apr floorplan. Put it after current_design $DESIGN
extract_physical_constraints pr_floorplan.def
```



If you don't use DEF file from APR Tool

- You can perform design floor planning directly from DC-G tcl script

```
# set urate
Set_utilization 0.8 #default 0.6

# set aspect ratio
set_aspect_ratio 2 #default 1.0

# set design boundary
create_die_area -coordinate {{0 0} {600 400}}

# set cell location
set_cell_location -coordinate {400 160} -fixed -orientation {N} RAM1

# create placement blockage
create_placement_blockage -coordinate {{350 110} {600 400}}

# create terminal
create_terminal -bbox {{0 40} {2 42}} -layer M1 -direction {left} -port A2

# report setting physical constraints
report_physical_constraints
```

N or 0: Nominal orientation (north)

S or 180: 180-degree rotation (south)

E or 270: 270-degree counterclockwise rotation (east)

W or 90: 90-degree counterclockwise rotation (west)

FN or 0-mirror: reflection in the y-axis (flipped north)

FS or 180-mirror: 180-degree rotation followed by reflection in the y-axis (flipped south)

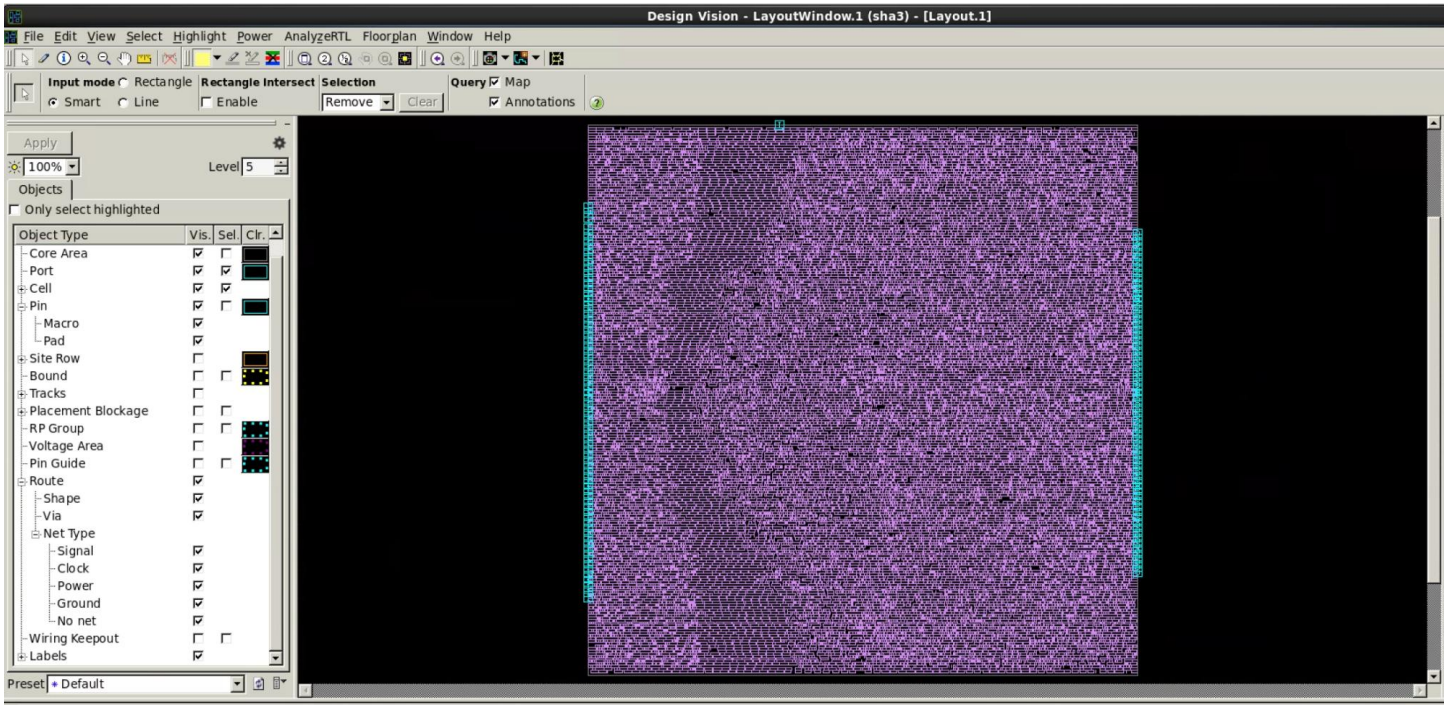
FE or 90-mirror: 90-degree counterclockwise rotation followed by reflection in the y-axis (flipped east)

FW or 270-mirror: 270-degree counterclockwise rotation followed by reflection in the y-axis (flipped west)



Visualization of Compiled Layout

- You can write out def (write_def dcg.def), write_ddc or write_floorplan for following APR



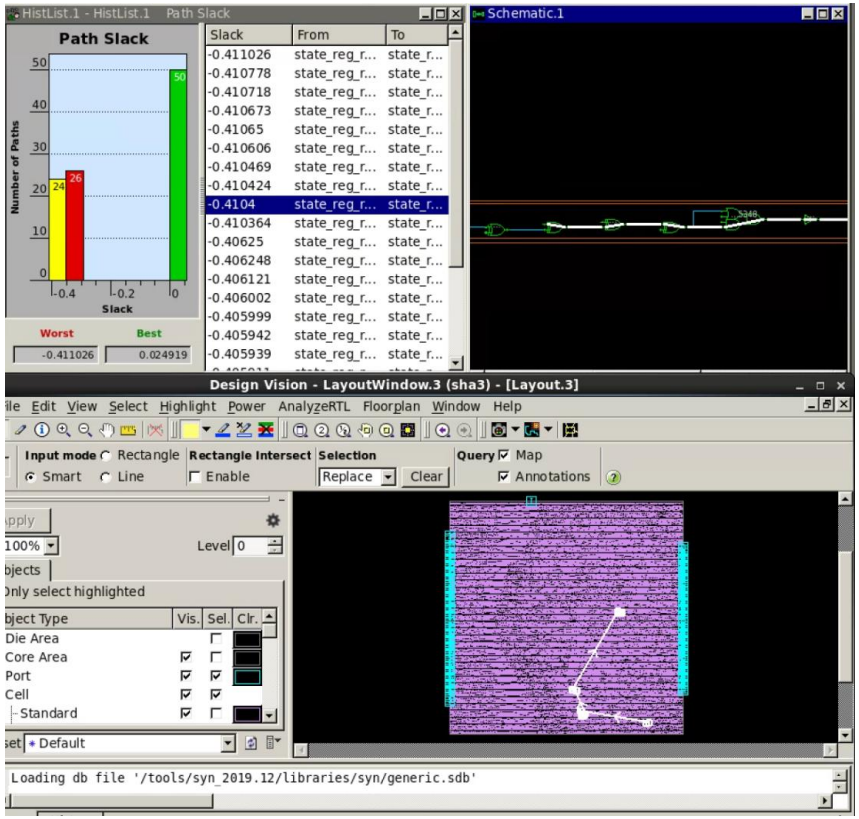
Visualize Timing Slack, the Path on Schematic and Layout

Schematic



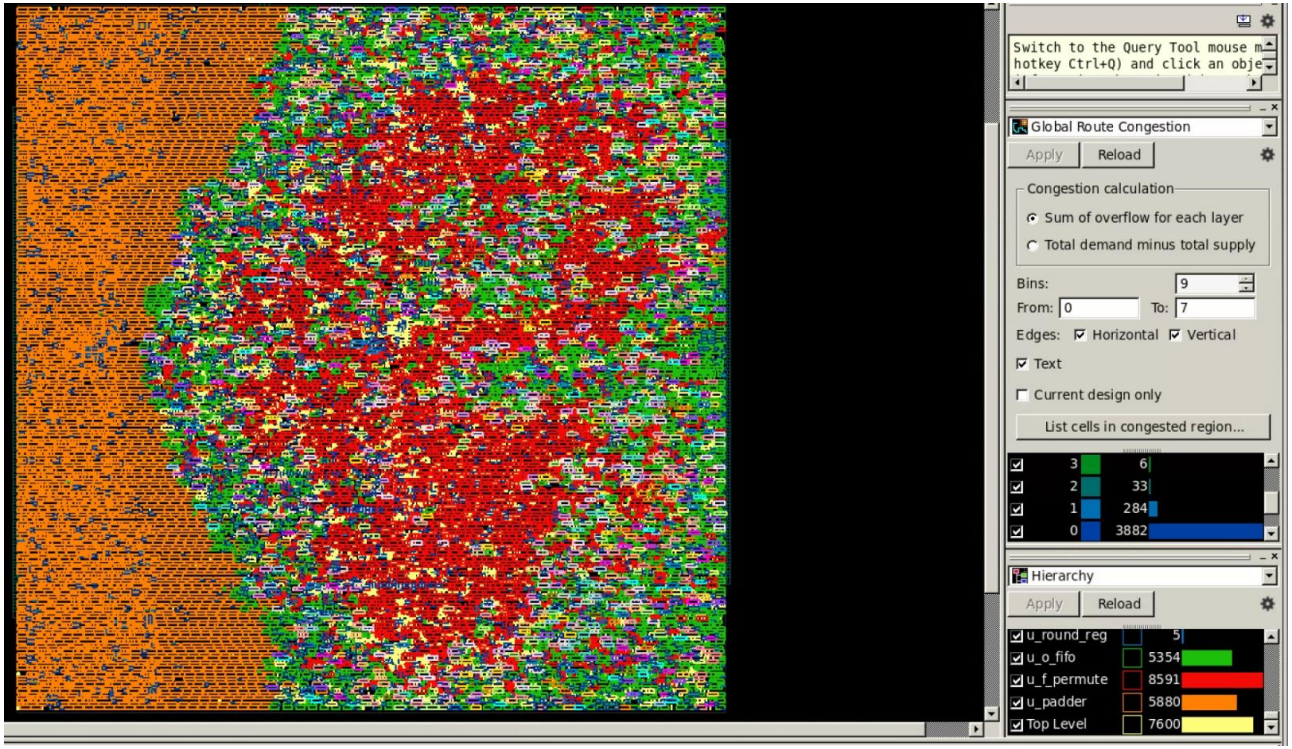
Timing Layout

- You can use design vision with topology mode to see the path timing and layout



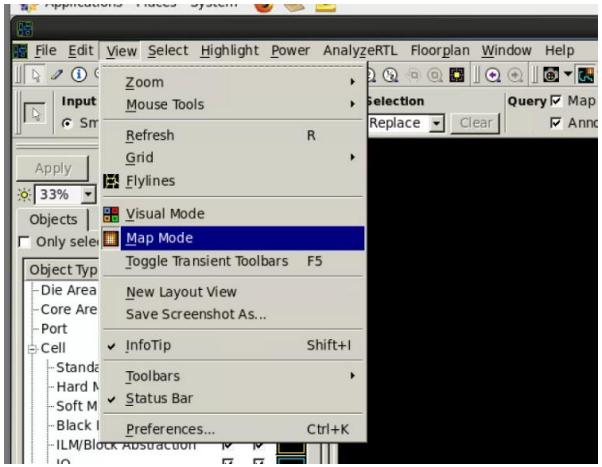
Coarse Placement Visualization

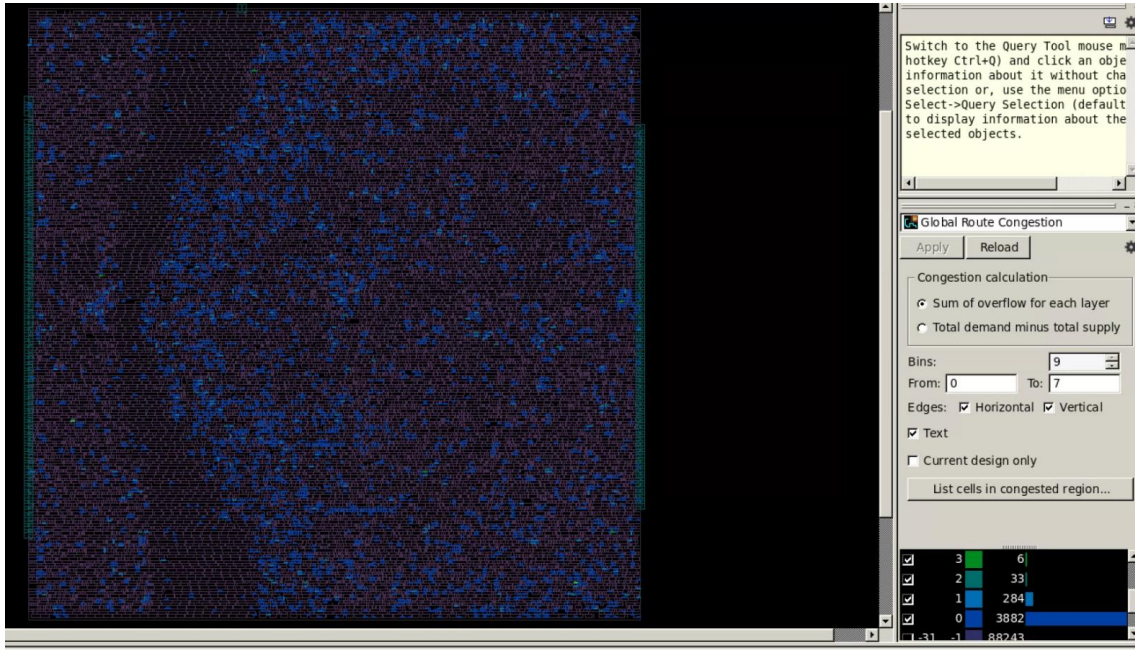
- Select Hierarchy → Press Reload



Global Route Congestion by SPG flow

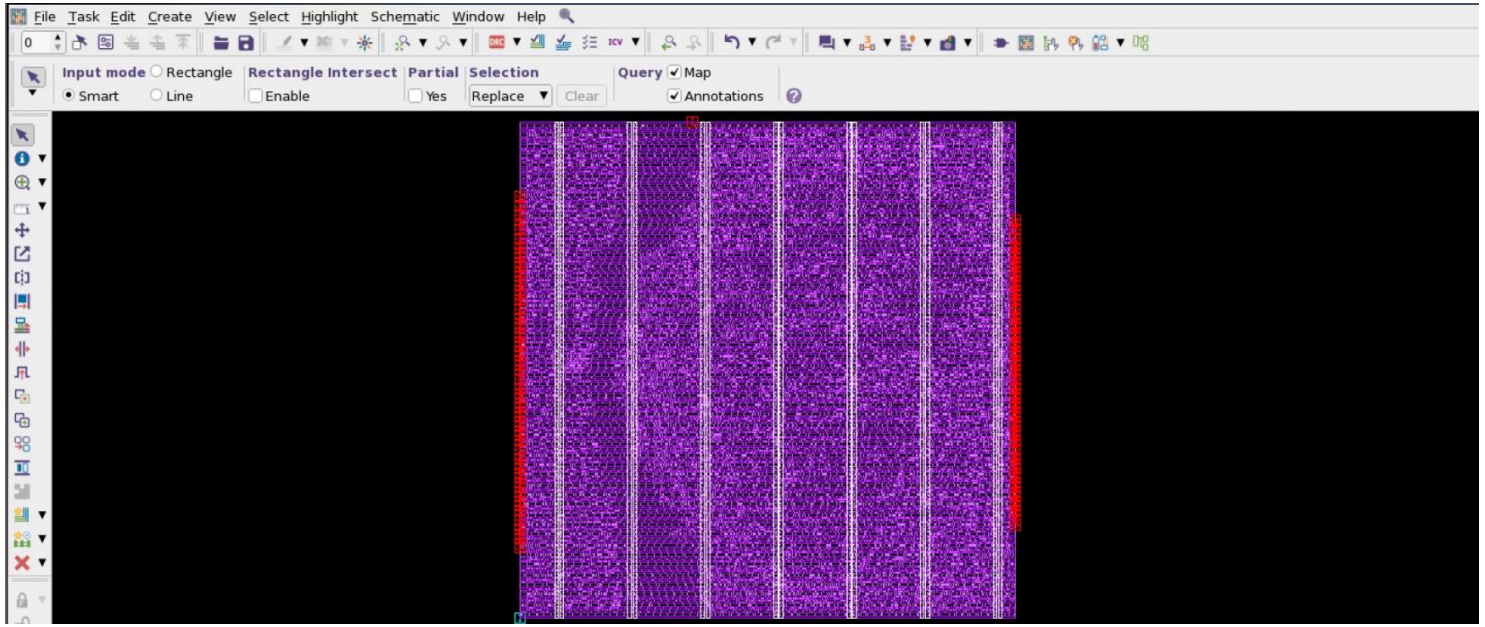
- View → Map Mode → Press Reload





Read DC-G Physical Floorplan to ICC-2/Innovus

- Method 1 (read_ddc to ICC-2 instead of read_verilog)
- Method 2 read_verilog and read_def to ICC-2
  (defIn in Innovus)
- Method 3 read_verilog and read_floorplan to ICC2





Outline

- Review on Previous Logic Synthesis
- Physical Synthesis
- **Low-power Design and UPF Flow**
- Low-power Design Case Study
- Conformal Low-power Check

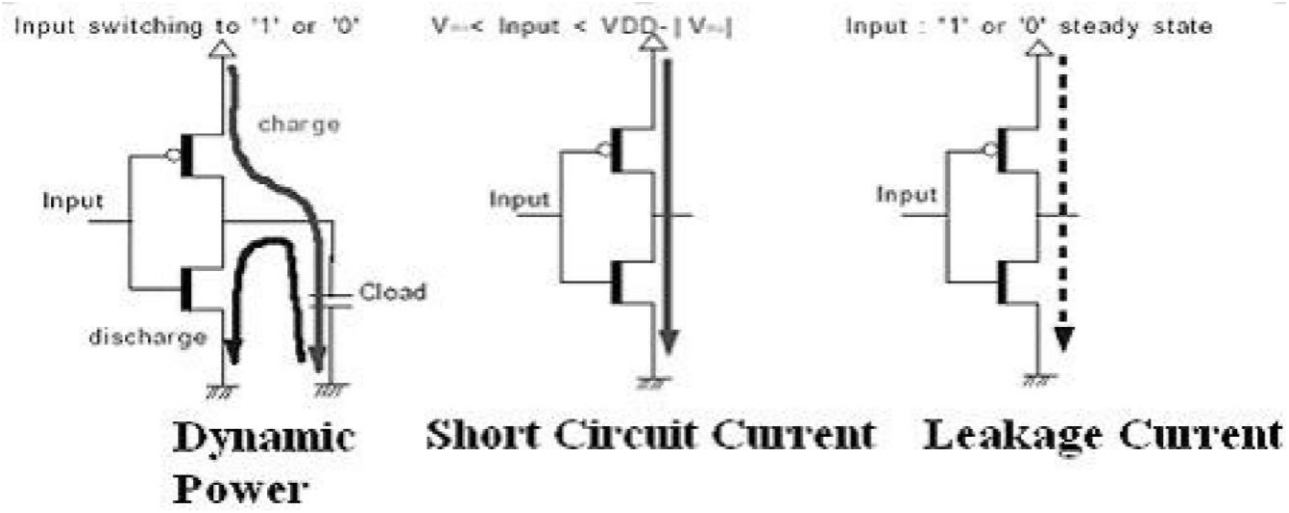


Important of Low-Power Design

- Cost factor of high-end systems
  - Reduce cooling and package cost
  - Reduce electricity burden
  - Enhance reliability of overheating
- Usability of portable systems
  - Smaller and lighter devices (low-capacity battery)
  - Longer battery lifetime
  - Longer device lifespan

Source of Power Dissipation

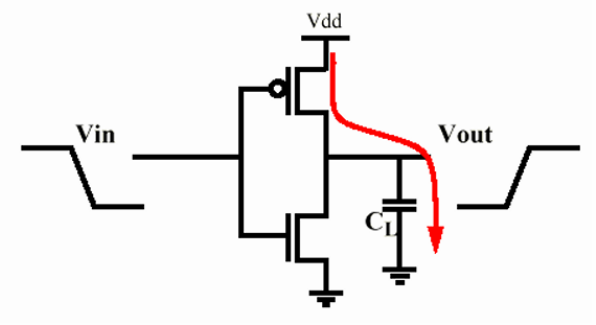
- Dynamic power: charging and discharge to capacitive load by transistor switching
- Short circuit current: conduction current when both PMOS and NMOS turn on simultaneously
- Leakage current: leakage by limited R_{off} of PMOS and NMOS



Basic of Dynamic Power

- The law of dynamic power consumption
  - Activity (α)
  - Capacitive Loading C_L
  - Supply voltage Vdd
  - Frequency f

$$P_{dyn} = \alpha C_L V_{dd}^2 f$$



- Mindset to reduce dynamic power
  - Reduce activity (Clock gating, Power Gating)
  - Reduce V_{dd} and f (Voltage Scaling)
  - Reduce C_L (Technology scaling)



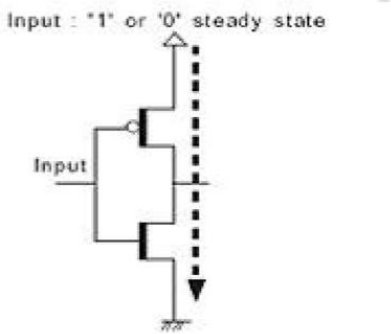
Basic of Static Power

- Leakage current depends on
  - Threshold voltage
  - W/L of MOSFET
  - Temperature (Reference temperature inversion for more detail)
    - T↑ and leakage↑ when process node <= 65nm and V_{DD} is low (< 0.8V)
    - T↑ and leakage↓ when process node > 65nm and V_{DD} is high (> 0.8V)
  - Architecture of MOSFET
  - Oxide thickness and channel doping concentration

Sub-threshold current

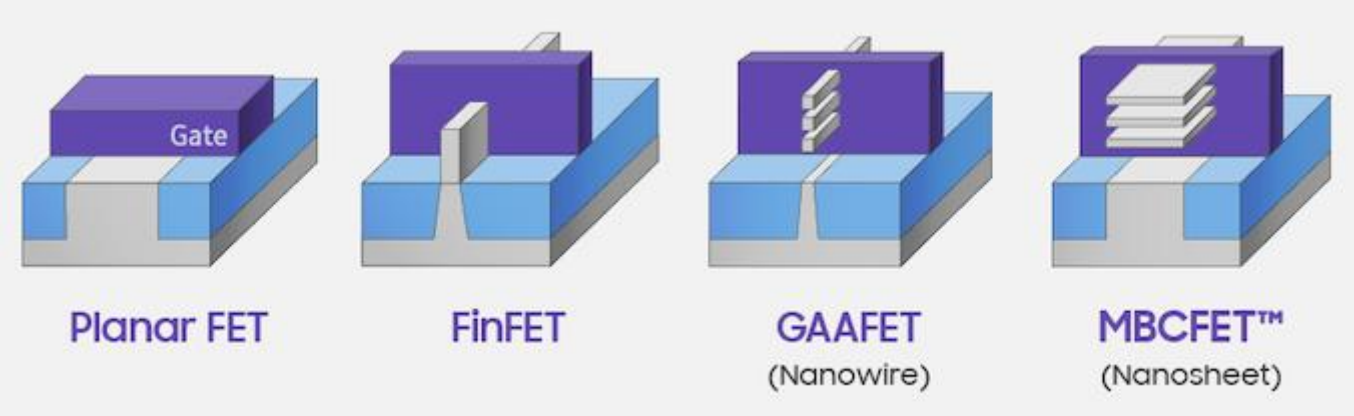
$$I_D = K \cdot e^{(V_{gs}-V_t)q/nkT}(1-e^{V_{ds}q/kT})$$

- Mindset to reduce static power
  - Increase Vt (Multi-Vt cell, body biasing…)
  - Increase W/L (Multi-channel length cell)
  - Reduce temperature (Cooling, power density)
  - Architecture (Planar → FinFet → GAA MOSFET)



Introduction of Different MOSFET

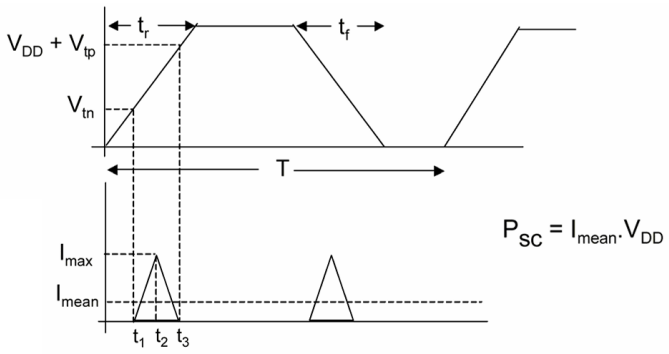
- The type of gate cover on MOSFET channel define different types of MOSFET architecture
  - Planar : one-side cover (22nm – 0.5um)
  - FinFET : Three-sided cover (N3(T), N7(I), N4(S) – 16nm)
  - GAAFET : Four-sided cover (< N2(T), < N3(S), < 20A (I))
- More cover side has more gate (leakage) control ability



Source from Samsung

Basic of Short Circuit Power

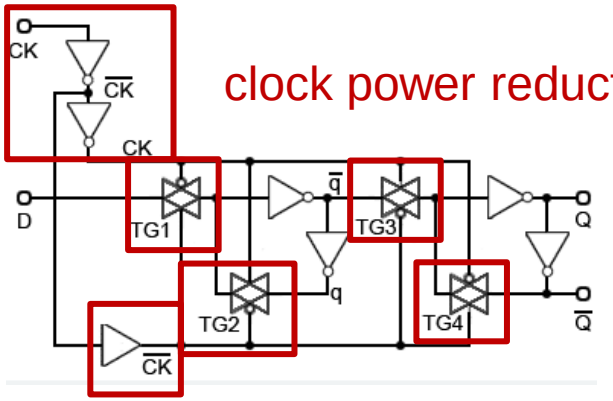
- Short circuit current depends on
  - Input slew (transition time) of gate
  - Output capacitive loading
  - Turn-on and turn-off speed of MOSFET (I_{on}/I_{off} ratio)



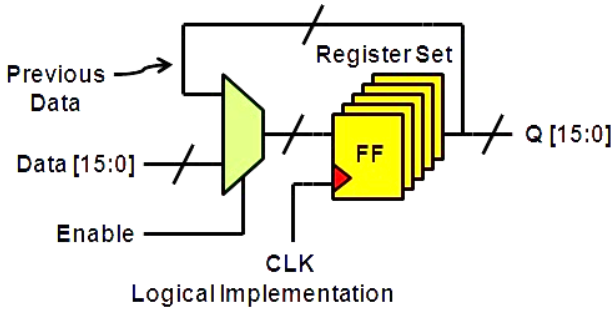
- Mindset to reduce short circuit power
  - Improve input slew (Use low-Vt cell or large cell as pre-driver)
  - Reduce output loading (Aware max fanout and max cap.)
  - Improve I_{on}/I_{off} ratio (By technology scaling and MOSFET Architecture)

Integrated Clock Gating Cell

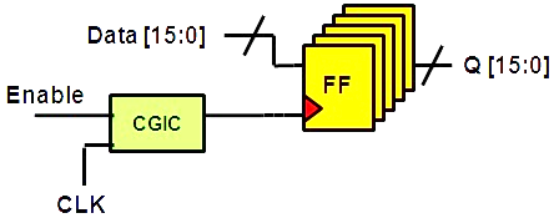
- Integrated clock gating cell (ICG) can reduce clock loading while data of FF is not toggling



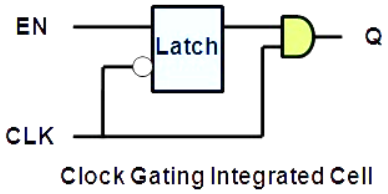
clock power reduction



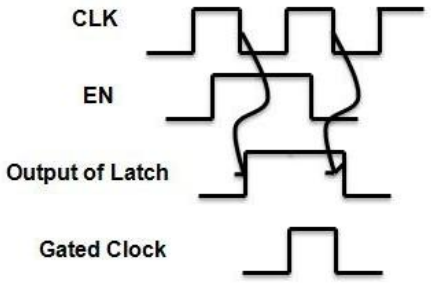
Logical Implementation



Low Power Implementation



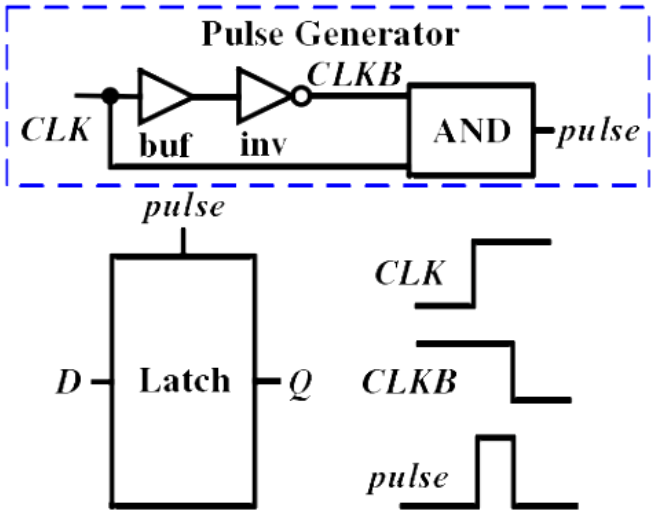
Clock Gating Integrated Cell



Low-Power Pulse-Latch Design

- Pulse-latch use pulse generator and latch to replace edge sensitive flip flop

- Area and clock loading of latch is 0.5x lower, which reduce significant power in the registers.

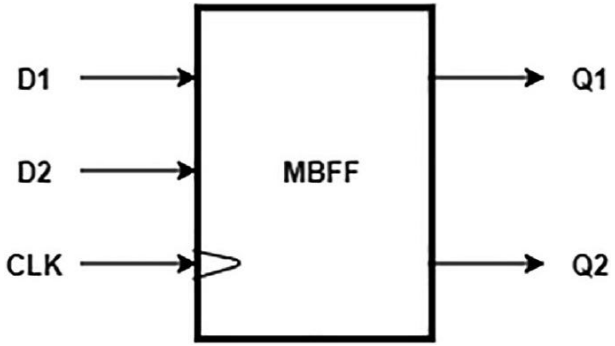




Multi-bit Flip Flop or Latch

- Multi-bit flip flop and latch reduce clock power by reducing their clock repeaters (repeater sharing in cell)
  - Can reduce clock routing resource in backend by common pin connection
  - Can reduce area by sharing clock repeater

- MBDFF and MBLAH usually have multiple cell height





In-compile MBDFF/LAH Insertion

- Workflow:
  - set_multibit_options before compile (Best to operate with DC-G)

```
1 set_multibit_options \
2     exclude_registers_with_timing_exceptions true \
3     ignore_timing_exception FALSE_PATH
```

  - Set the multibit option to timing driven

```
dc_shell> set_multibit_options -mode timing_driven
```

  - After compiling you can use report_multibit_banking to see the multibit register usage

```
Total number of sequential cells:                          2422
    Number of single-bit flip-flops:                       1699
    Number of single-bit latches:                            10
    Number of multi-bit flip-flops:                         711
    Number of multi-bit latches:                              2
Total number of single-bit equivalent sequential cells:    3147
    (A) Single-bit flip-flops:                             1699
    (B) Single-bit latches:                                  10
    (C) Multi-bit flip-flops:                              1434
    (D) Multi-bit latches:                                    4
Sequential cells banking ratio ((C + D) / (A + B + C + D)):  45.69%
Flip-Flop cells banking ratio  ((C) / (A + C)):              45.77%
Multi-bit Register Decomposition:
```



MBDFF/LAH Insertion in RTL-Level

- Workflow:
  - Make sure the Mbit cell is available in the lib
  - Use // synopsys infer multibit “reg_name” to infer MBDFF

```
//synopsys infer_multibit "q0"
always @(posedge clk)begin
  if (!rst) q0 <= 0;
  else q0 <= d0;
end
```

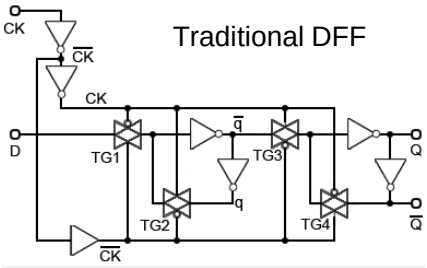
  - Set hdlin_infer_multibit in sdc to
    - default_none: only map MBDFF with synopsys infer_multibit
    - default_all: automatically infer MBDFF by logic analyzing

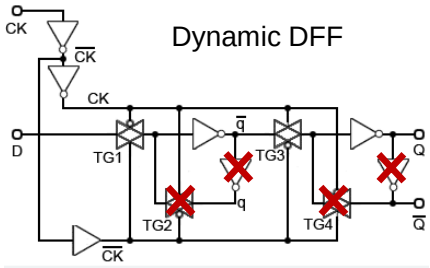
Register Name	Type	Width	Bus	MB	AR	AS	SR	SS	ST
q0_reg	Flip-flop	8	Y	Y	N	N	N	N	N



Dynamic D-Flip Flop and Latch

- Dynamic FFs and latches "do not" have feedback loop in their internal latch
- Only gate capacitances in data storage node are used for keeping data
- Dynamic FFs and latches
  - Have minimum operating frequency (leakage issue)
  - Will loss data when apply clock gating (system must be none stop)
  - Have < 0.5X lower area and < 0.5X lower power
  - Is sensitive to noise and SI crosstalk

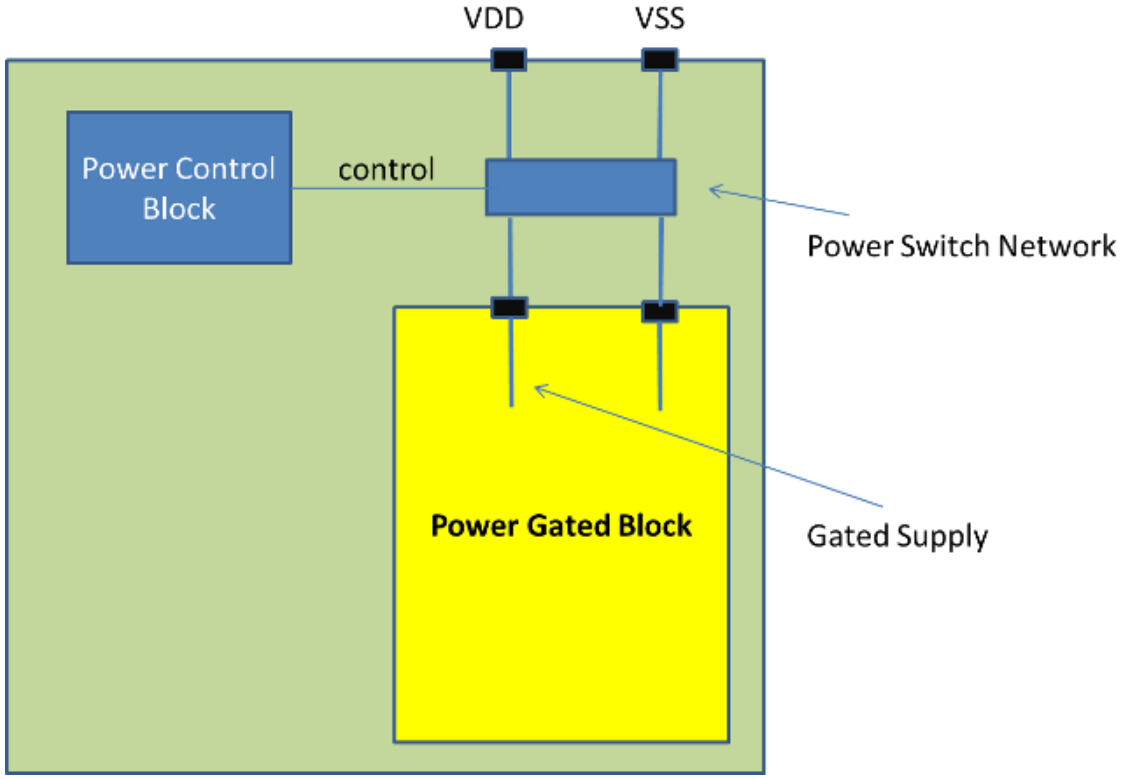






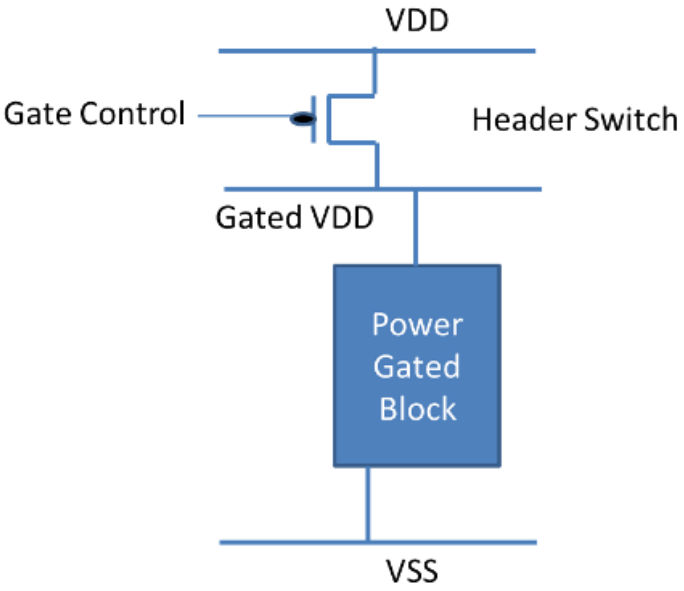
Power Gating

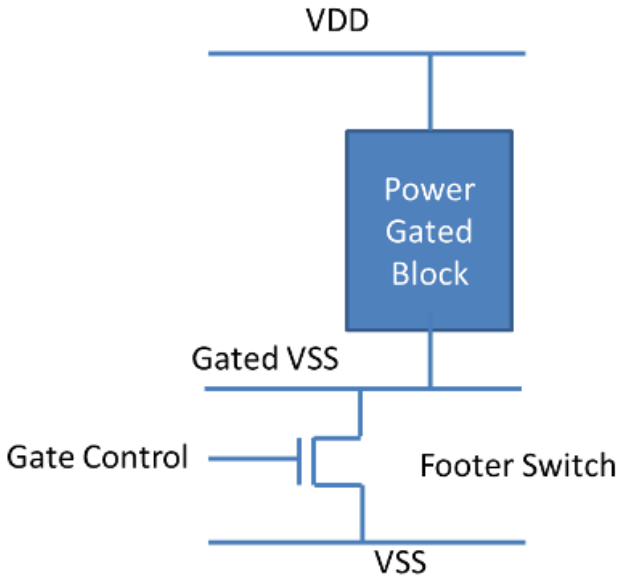
- Power gating is used to cut off the power or ground supply to enable a deep sleep for blocks
- Reduce significant dynamic and static power



Power Switch

- Typically two kind of power switches can be used
  - Header (PMOS) is used to cut off VDD (Smaller leakage, higher area, higher V drop)
  - Footer (NMOS) is used to cut of VSS (Higher leakage, smaller area, lower V drop)
- Overhead:
  - Power switch area (~10%), excessive power routing resource
  - Virtual VDD and VSS has 10~20mV voltage difference (speed degradation)







Multi-Vt Gate Length Standard Cell

- Longer channel length cells have lower leakage and speed
- Higher threshold voltage cells have lower leakage and speed

- For TSMC 28nm Arm standard cell
  - Postfix R, L, H indicate Regular, Low, High threshold cells
  - 30, 35, 40 indicate gate length of 30, 35, 40nm
  - Inverter: INV_X1M_A9TR30



Leakage Power Optimization - 1

- **Multi-threshold voltage (V_t)** libraries to **reduce leakage power**
  - Low V_t cells: higher speed, higher leakage
  - High V_t cells: lower speed, lower leakage
- Note: Different doping concentration for high & low V_t MOSFETs
  - High V_t: Lower source-drain current → slower, lower leakage

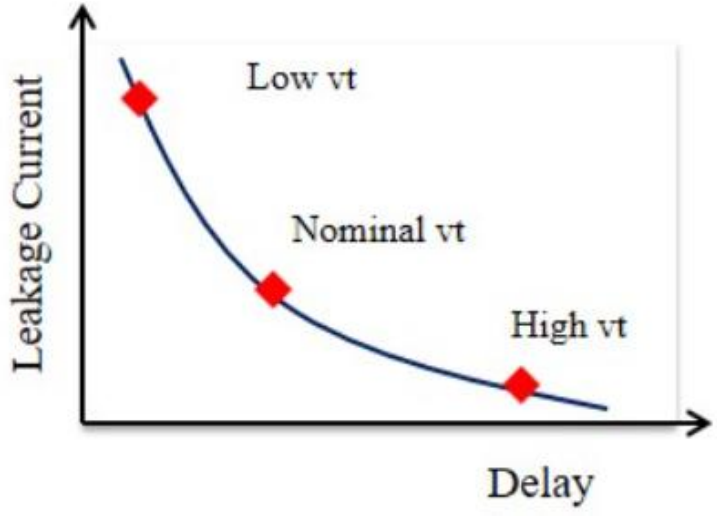
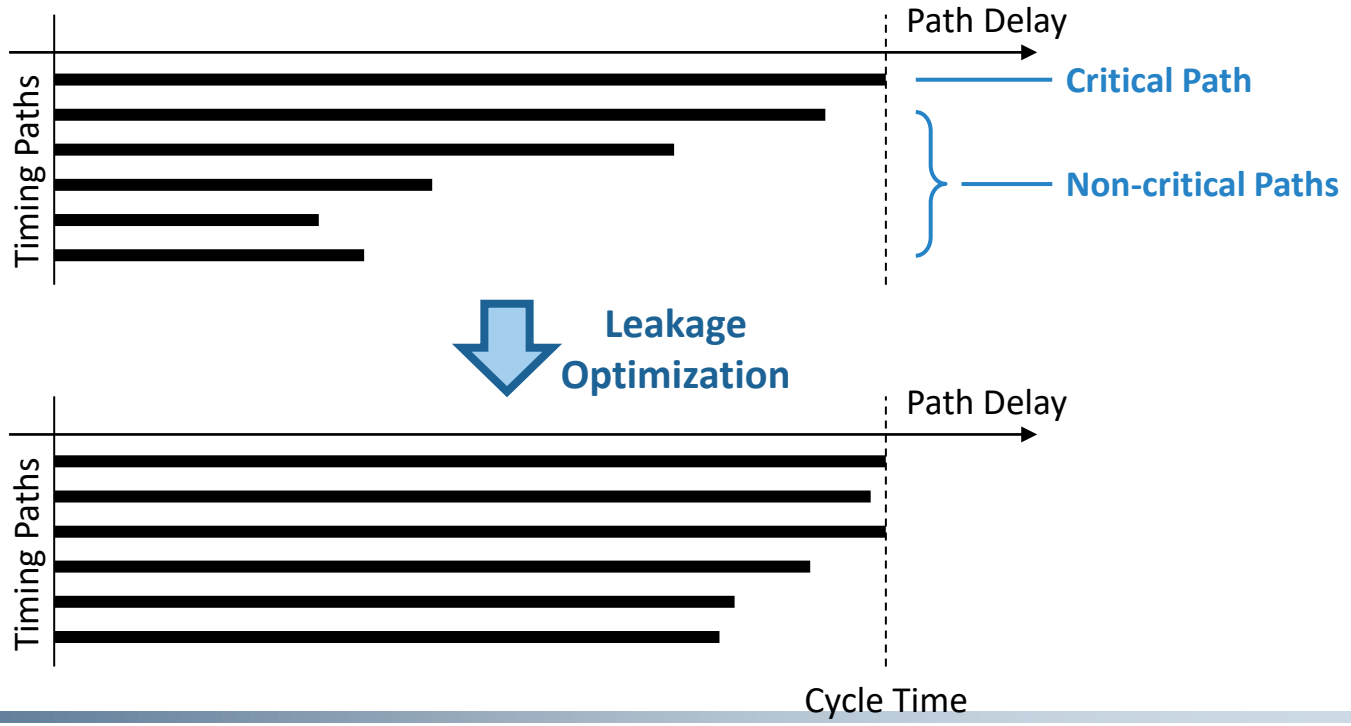


Image source: https://www.vlsiguru.com/low-power-pavan/



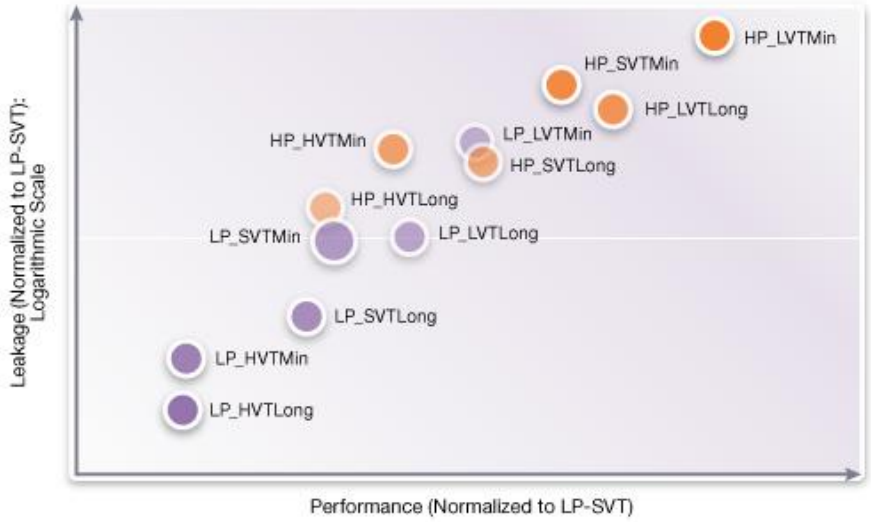
Leakage Power Optimization - 2

- Trade-off between timing & power
  - Use **low V_t** cells on critical paths to **improve timing**
  - Use **high V_t** cells on non-critical paths to **save power**
- Usually optimize leakage power after timing optimization



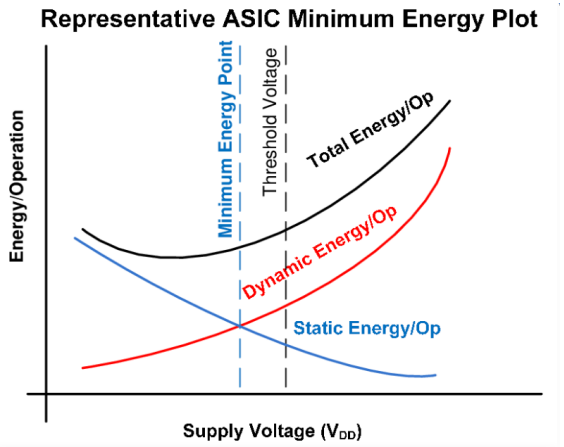
Multi-Vt Gate Length Case Study

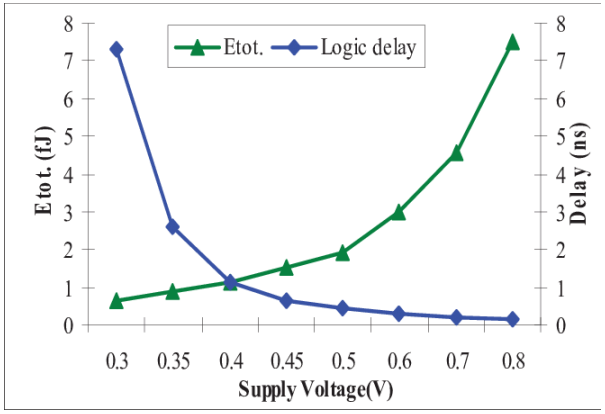
- Synopsys 180nm-3nm stdcell have multi-threshold feature
- For 40nm below, multi channel length stdcells are available
- It is noted that you "**cannot**" use above **four** types of Vt-cell in **TSMC** process



Dynamic Voltage Frequency Scaling

- The energy can be scale down when apply dynamic voltage frequency scaling (DVFS).
  - Dynamic power is proportional to V^2 while static power is linear or nearly constant
  - Logic delay is nearly linear when VDD >> V_{th} voltage while growing exponentially around V_{th}.
- Need high performance (Fmax↑, VDD↑)
- Need low power consumption and low energy (Fmax↓, VDD↓)



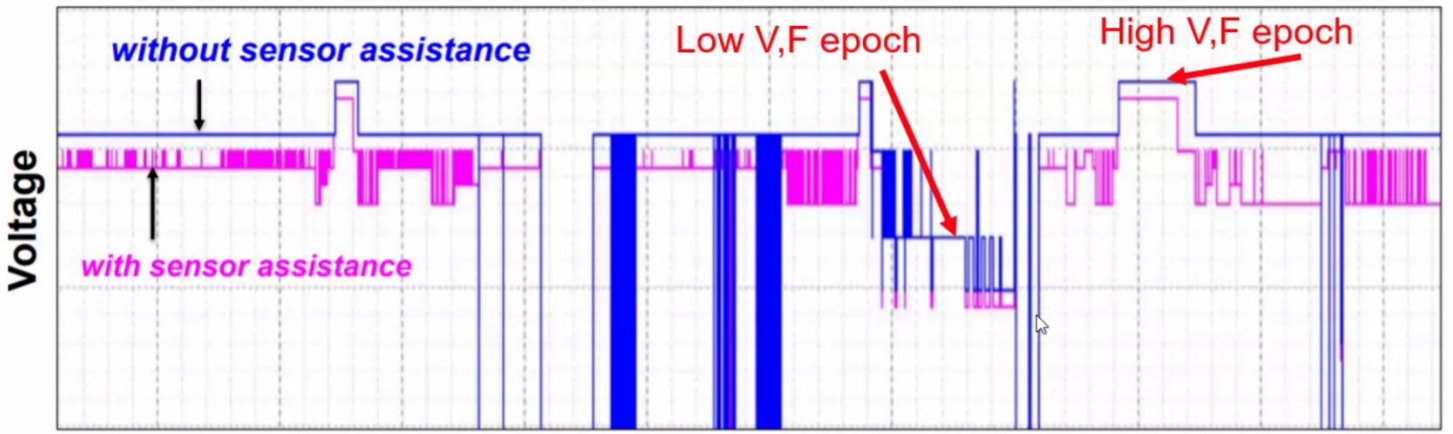




Adaptive Voltage Scaling-Case Study1

- This work is from MTK's smart sensor application on ISSCC-2021
  - On chip **PVT sensor** and workload prediction function for smart AVS
  - ~15% energy efficiency gain in system level

- Adapt frequency/voltage to performance demands of workload
  - Exploit natural variations in performance demands to avail efficiency gains
  - Deliver higher-performance at higher power-point when required
  - Most workloads require lower-performance availing efficiency benefits



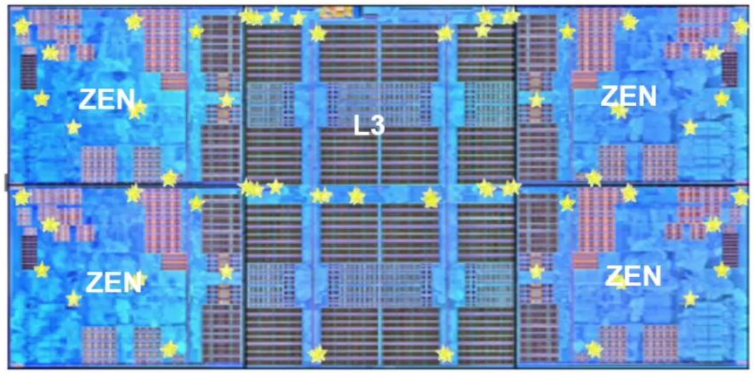
Huang et al., ISSCC 2021 [38], Copyright © 2021 IEEE



AVS-Case Study 2

- AMD Zen series have thousands of low-cost digital workload monitor that can be use to track and predict AVS strategy

"Sense-and-Adapt" System Example : AMD Zen Series



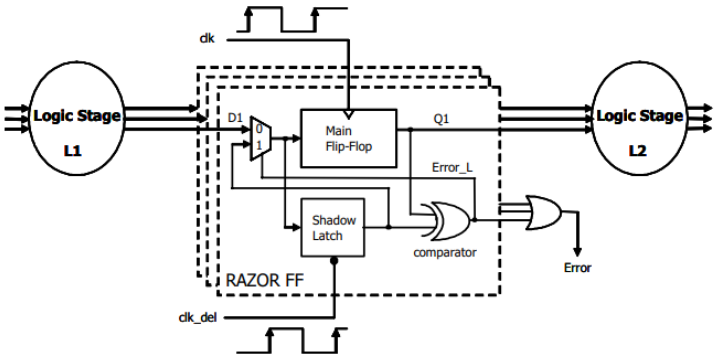
T. Singh et al., JSSC 2018, ISSCC 2020 [39][40]

- >1300 critical-path monitors
- 48 on-die high bandwidth power-supply monitors
- 20 thermal diodes
- 9 high-speed droop detectors

- Numerous sensors to measure frequency, voltage, power and temperature
- Sensors drive characterization, voltage and frequency tuning and adaptive-clocking

Timing Tolerant DFF (Razor-FF)

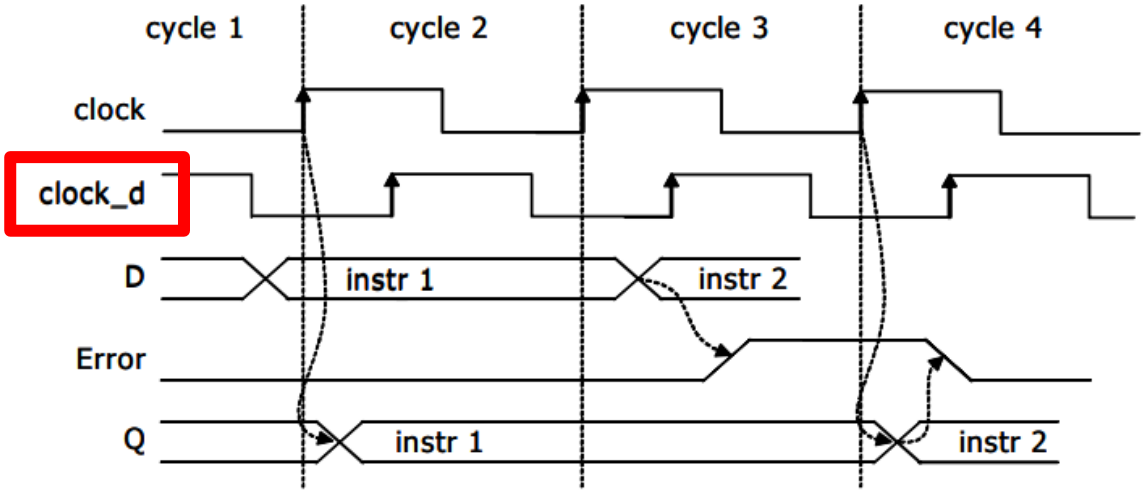
- Razor DFF is a self-timing calibration register that can be apply to critical path to improve Fmax
  - It use shadow latch to perform timing borrowing
  - It calibrate data while timing violation is occurred (Data in Main FF is different with the one in shadow latch)
- Additional pipeline stall (clock gating) or replay are used for auto-calibrate timing violation
  - Fmax is boosted up → You can apply DVFS more by lowering more VDD





Razor Timing Diagram

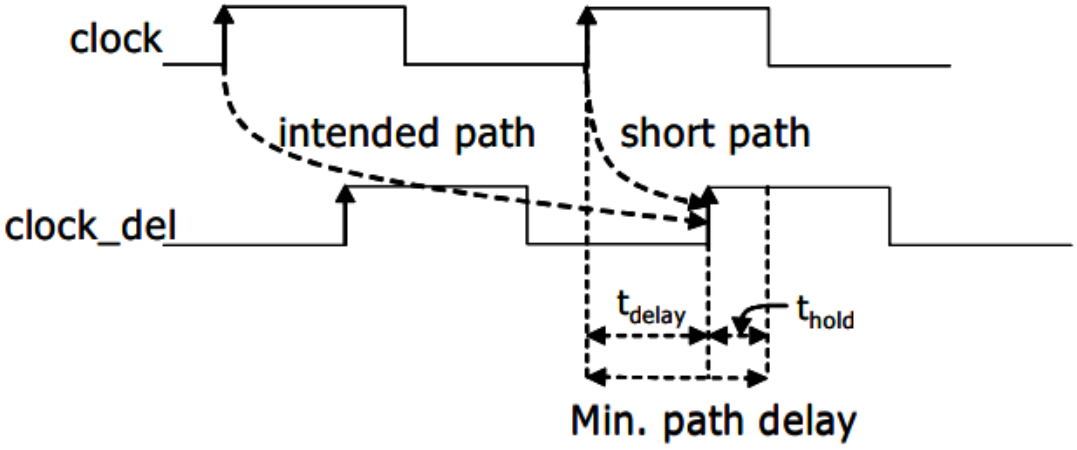
- Delayed clock trigger the shadow latch
- Compare to the DFF, the latch has **more setup time** due to time borrowing
- As the result, we can ensure the latch always trigger right value
- The error signal compares the two triggered value, and raised the error flag.
- Third cycle needs to be **stalled or flushed out**





Min. Path Issue of Razor

- The hold time constraint would be more severe
- If the min path delay is too short, **the triggered right value would be flushed out**
- Additional delay buffers also add some **power overhead**



Min. Path Delay > t_{delay} + t_{hold}

Razor FF IEEE MICRO-2024

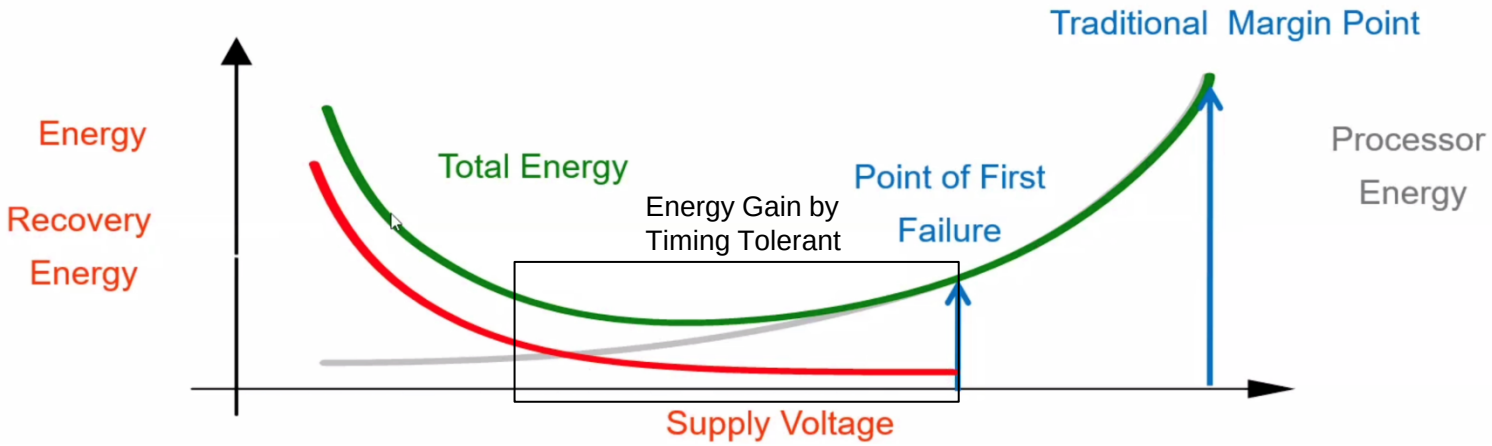
[Public]

Error-Resilient Computing using Razor

MICRO Test of Time Award, Ernst, Das et al. IEEE Micro, 2004 [43]

Key idea: Exploit the dynamic nature of variations

- Speculatively operate without full setup margin
- Explicitly check for late-arriving signals
- In the event of a timing error, invoke system recovery mechanism
- Adapt VDD/CLK to target near-zero error-rate operation

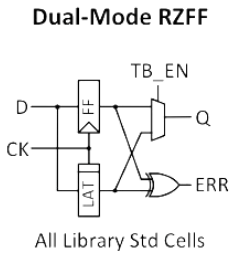


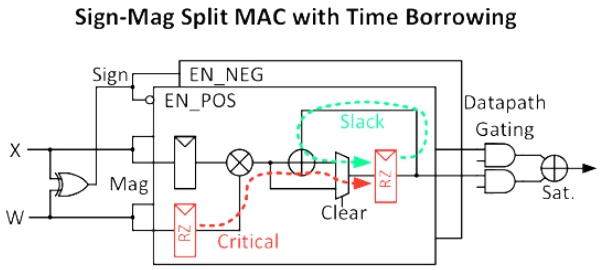


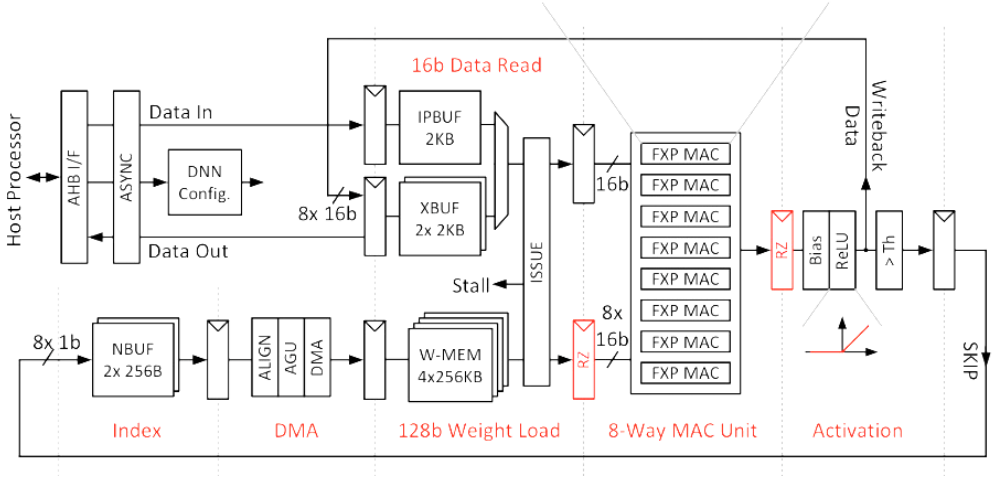
Case Study – AI Accelerator with Razor

- This work is published in Paul N.W. et al., ISSCC-2017
- It applies RZFF in MAC to get significant performance gain
  - Fmax 667MHz → 1.2GHz
  - VDD scale to 0.715V while maintain original sign-off frequency

Process Tech.	TSMC 28nm HPC 1P10M
Die Size	2.4mm x 2.4mm
Total SRAM	SoC: 128KB / W-MEM: 1MB / DNN-Engine: 6.5KB
Total FFs	8460 (896 RZFFs)
ML Model	FC-DNN Classifier
Weight Precision	8-bit / 16-bit Fixed-Point
Native Model Size	Hidden Layers: 0-6 Nodes/Layer: 1-1024
Error Tolerance	$>10^{-1}$ @ 98.36% Accuracy
Supply Voltage	0.9V (nom.) 0.6 – 1.1 V (operational)
F_{MAX}	667MHz @ 0.9V 1.2GHz @ 0.9V (w/ Razor)
Power Consumption	33.7mW @ 667MHz/0.9V 22.4mW @ 667MHz/0.715V 63.5mW @ 1.2GHz/0.9V
Leakage	3.03mW @ 0.9V
RZFF Overhead	3.24x area per RZFF





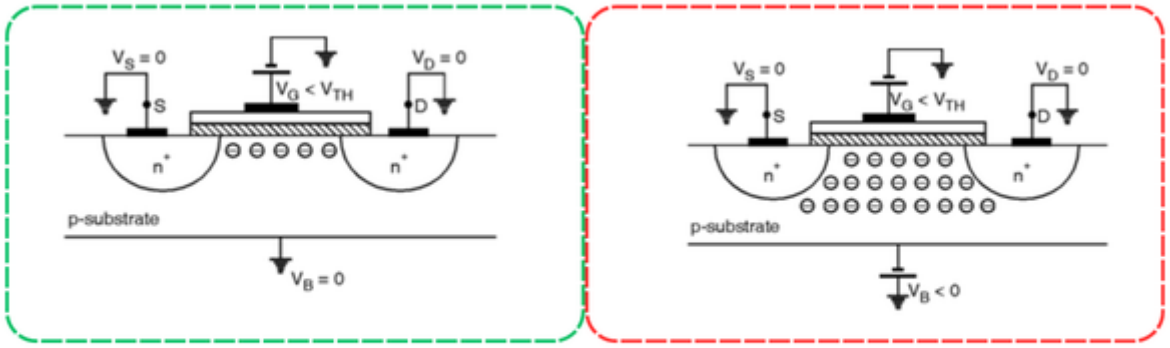


Backward Body Biasing

- Backward biasing is used in SOI or deep N-well technology
- Body biasing ($V_{SB} > 0$) of MOSFET reduce static leakage while the circuit is in retention or deep sleep

$$V_{TN} = V_{TO} + \gamma\left(\sqrt{V_{SB} + 2\phi} - \sqrt{2\phi}\right),$$

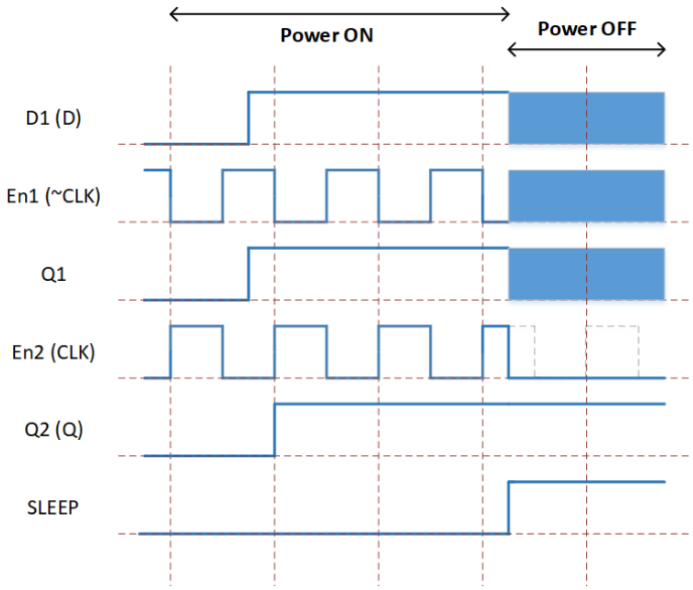
V_{TO}是基極與源極之間無電位差時的臨界電壓，γ是基板效應參數，2ϕ則是與半導體能階相關的參數（禁帶中線與費米能級的差值）。

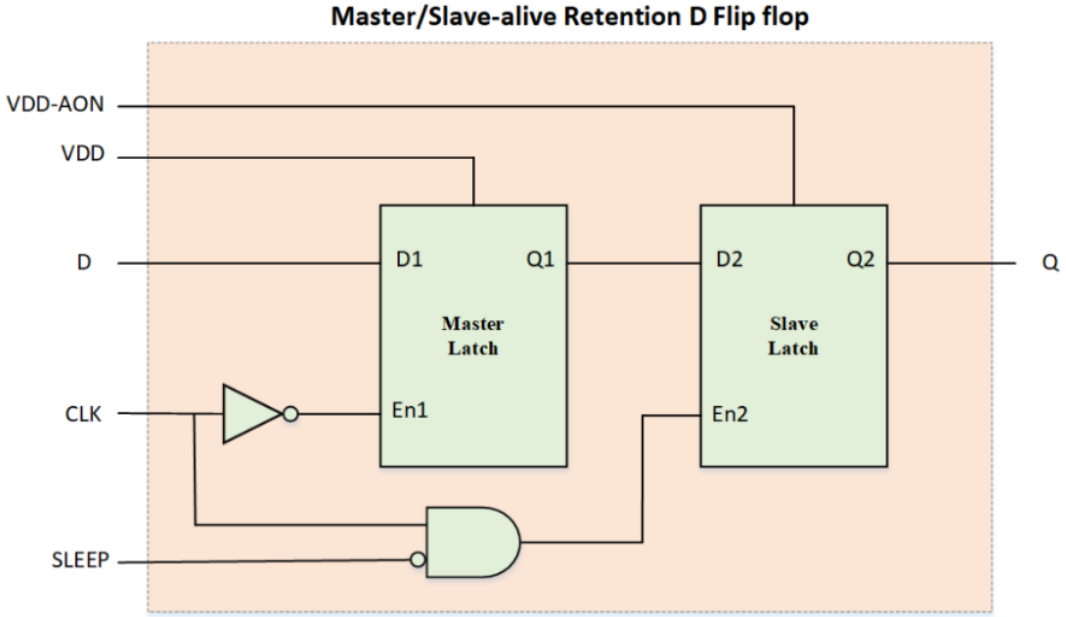


No Body Effect　　Body Effect

Retention Flip Flop

- Retention Flip Flop use secondary power in slave latch to keep data when apply power gating
- Double cell height, and may introduce additional power routing penalty

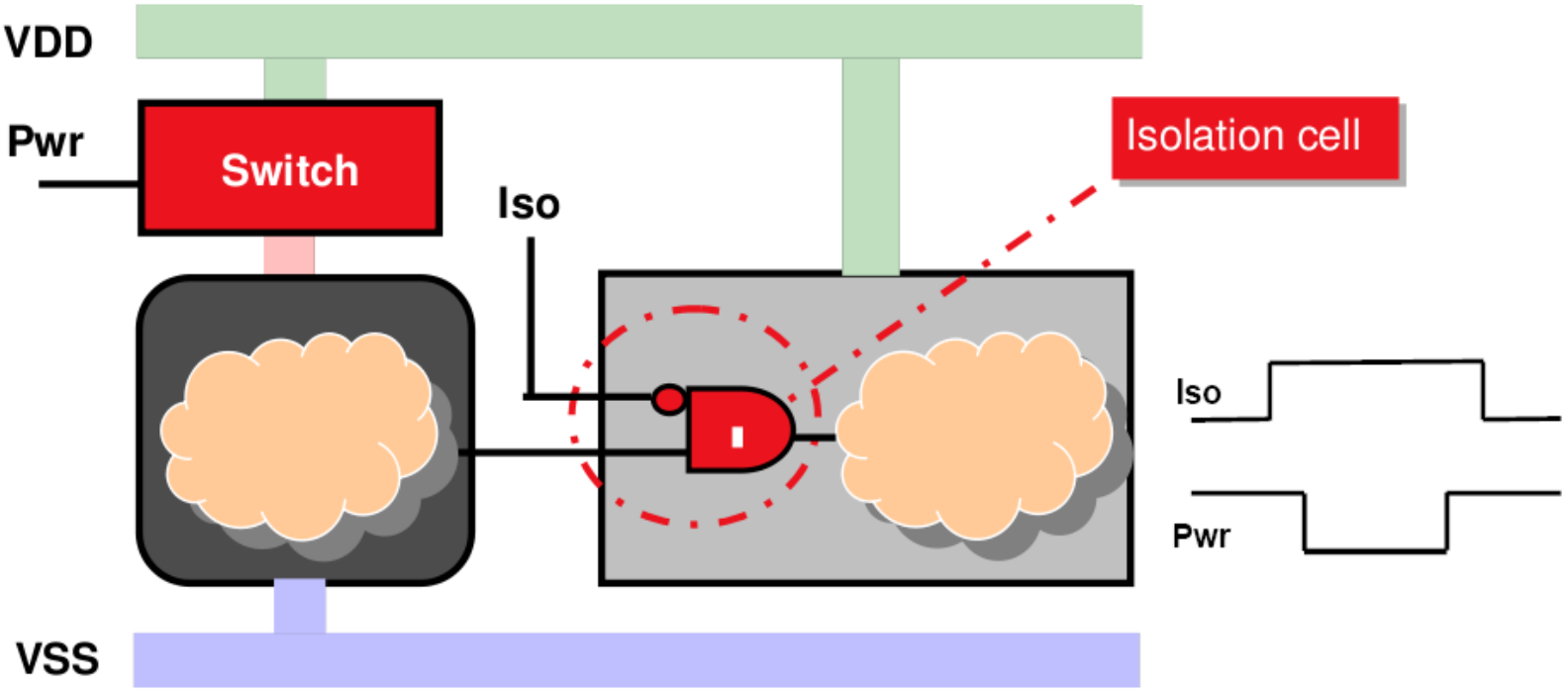




Isolation Cell

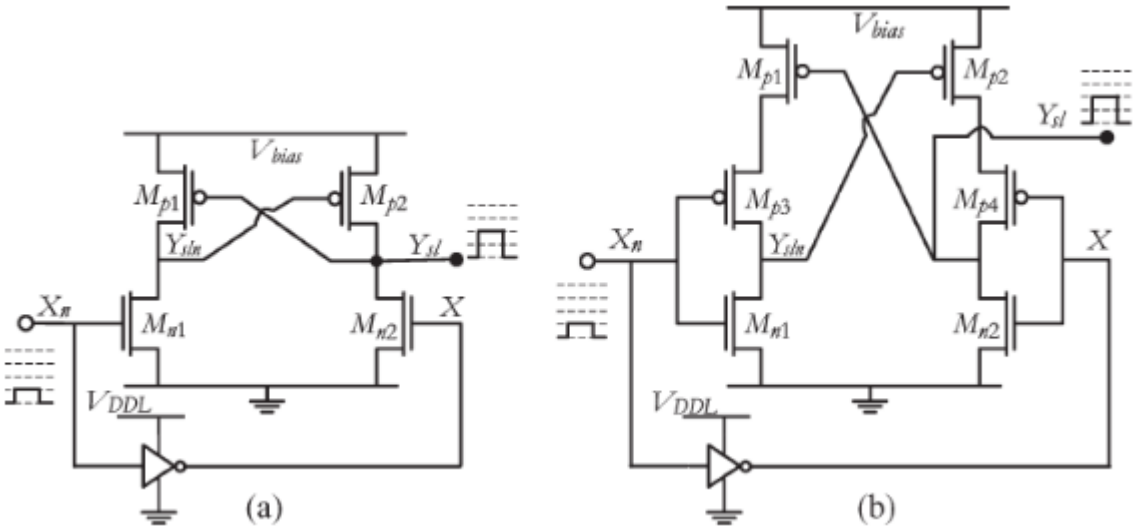
- Isolation cell is used to prevent non-deterministic value produced by the sleep power domain





Level Shifter (LVL) Cells

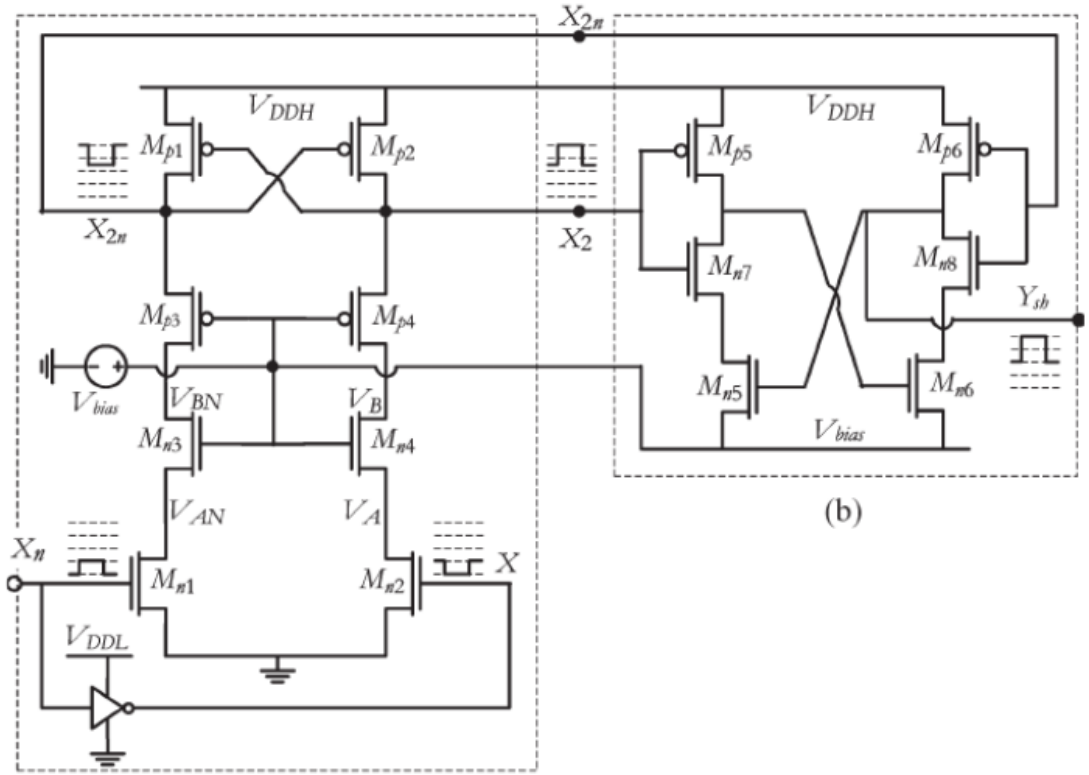
- Level shifter cells are used to convert signals from a low-power domain to a high-power domain, or vice versa.
- Two level shifter design is shown below
  - (a) the simplest level shifter
  - (b) add Mp3 and Mp4 to speed up the conversion
- High to low LVL is not needed if the difference of VDDH and VDD is not high.





High Voltage Swing Level Shifter Cells

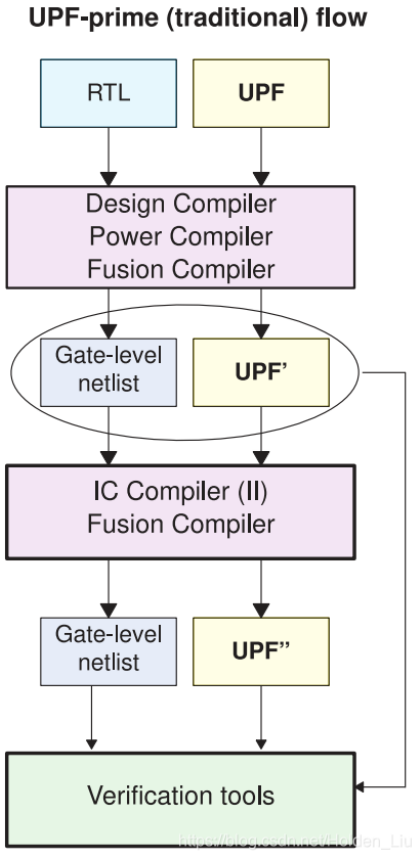
- Use Vbias to bias two back to back N/PMOS. (Prevent oxide breakdown in $M_{p1,2}$ and $M_{n1,2}$)
  - X_2/X_{2n} voltage cannot have full swing





Unified Power Format (UPF)

- The Unified Power Format (UPF) is a standardized format used for describing power management in integrated circuit (IC) designs.
- Power Domain
  - Defines the behavior of different sections of the circuit under different power modes
- Power state
  - Describes the behavior of the circuit under different power modes, including active, sleep and deep sleep states
- Power Intent
  - Describes the power management goals such as power consumption limits, power gating, and power mode transitions.





UPF Version History

IEEE 1801-2013 (UPF-2.1)

Key:
- Accellera UPF-1.0 (2007)
- IEEE 1801-2009 (UPF-2.0)
- IEEE 1801-2013 (UPF-2.1)
- Deprecated/Legacy

Navigation:
- set_scope
- set_design_top

Supply Nets:
- create_supply_port
- create_supply_net
- connect_supply_net
- create_power_switch

Power Domains:
- create_power_domain
- set_domain_supply_net
- create_composite_domain

Power States:
- add_port_state
- create_pst
- add_pst_state
- add_power_state
- describe_state_transition

Simstates:
- add_power_state
- set_simstate_behavior

Attributes:
- set_port_attributes
- set_design_attributes
- HDL and Liberty attributes

Supply Sets:
- create_supply_set
- supply set handles
- associate_supply_set
- connect_supply_set
- set_equivalent

Strategies:
- set_repeater
- set_retention_elements
- set_retention
- set_retention_control
- set_isolation
- set_isolation_control
- set_level_shifter

Implementation:
- map_retention_cell
- map_isolation_cell
- map_level_shifter_cell
- map_power_switch_cell
- use_interface_cell

Control Logic:
- create_logic_port
- create_logic_net
- connect_logic_net

HDL Interface:
- bind_checker
- create_hdl2upf_vct
- create_upf2hdl_vct

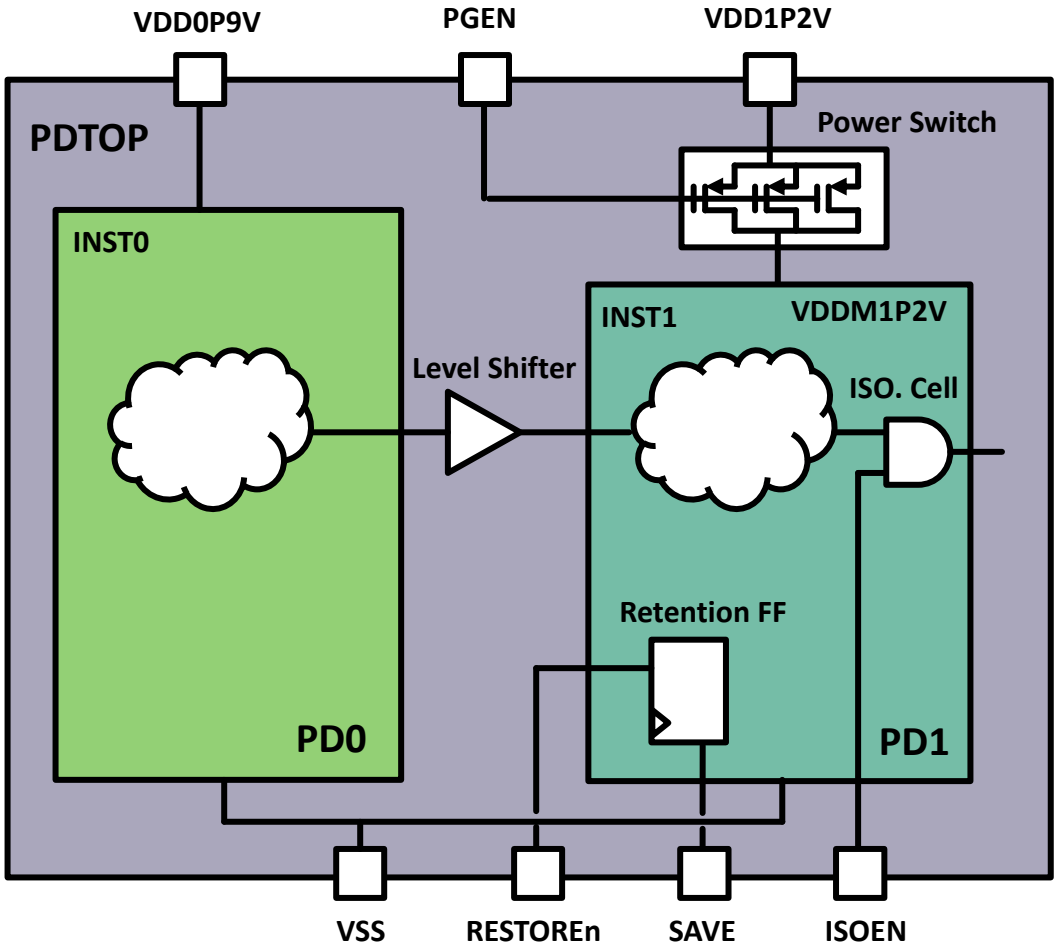
Code Management:
- upf_version
- load_upf
- save_upf
- load_upf_protected
- load_simstate_behavior
- find_objects
- begin_power_model
- end_power_model
- apply_power_model

Power Management Cells:
- define_always_on_cell
- define_diode_clamp
- define_isolation_cell
- define_level_shifter_cell
- define_power_switch_cell
- define_retention_cell

IEEE STANDARDS ASSOCIATION IEEE



An UPF Example





Create Supply Port, Net and Set

- Create supply port to top domain

```
create_supply_port  VDD0P9V
create_supply_port  VDD1P2V
create_supply_port  VSS
```

- Create supply net to top domain

```
create_supply_net  VDD0P9V
create_supply_net  VDD1P2V
create_supply_net  VSS
create_supply_net  VDDM1P2V
```

- Connect supply port to supply net

```
connect_supply_net  VDD0P9V -ports VDD0P9V
connect_supply_net  VDD1P2V -ports VDD1P2V
connect_supply_net  VSS -ports VSS
```

- Create supply set

```
create_supply_set SS0 -function {power VDD0P9V} -function {ground VSS}
create_supply_set SS1 -function {power VDD1P2V} -function {ground VSS}
create_supply_set SS2 -function {power VDDM1P2V} -function {ground VSS}
```



Create Power Domain

- Power domain should contain scope, elements and supply set
- By default, three supply set are included
  - Primary
  - Default retention
  - Default isolation

```
Syntax:
create_power_domain
    domain_name
    [-elements cells]
    [-exclude_elements cells]
    [-include_scope]
    [-scope instance_name]
    [-supply {supply_set_handle_name [supply_set_name]}]*
    [-available_supplies {supply_set_name_list}]
```

```
Ex:
create_power_domain PDTOP \
    -supply {primary SS0} –available_supplies {SS1}
create_power_domain PD0 –elements {INST0} –scope {INST0} \
    -supply {primary SS0}
create_power_domain PD1 –elements {INST1} –scope {INST1} -supply \
{primary SS2} –available_supplies {SS1}
```



Add Power State

- Create power state for supply set

```
Syntax:
add_power_state
   [-supply | -domain] object_name
   [-state state_name {[-supply_expr {supply_expression}]
                      [-logic_expr  {logic_expression}]
                      [-simstate    {simstate}]
                      [-illegal]}]*
```

```
Ex:
add_power_state –supply SS0 –state ON {-supply_expr \
{power == {FULL_ON 0.9} && ground == {FULL_ON 0.0}}}
add_power_state –supply SS1 –state ON {-supply_expr \
{power == {FULL_ON 1.2} && ground == {FULL_ON 0.0}}}
add_power_state –supply SS2 –state ON {-supply_expr \
{power == {FULL_ON 1.2} && ground == {FULL_ON 0.0}}} \
-state OFF {-supply_expr {power == {OFF} && ground == {FULL_ON 0.0}}}
```



Create Power Switch

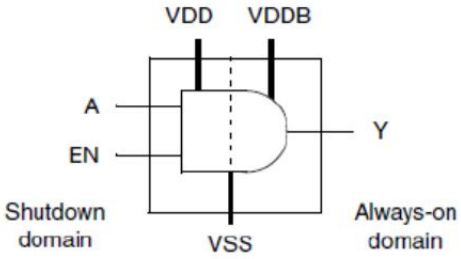
```
Syntax:
create_power_switch
                switch_name
                -domain domain_name
                -output_supply_port {port_name supply_net_name}
                -input_supply_port  {port_name supply_net_name}
                -control_port {port_name net_name}
                -supply_set supply_set_name
                [-ack_port {port_name net_name [{boolean_function}]}]
                [-ack_delay {port_name delay}]
                -on_state {state_name input_supply_port {boolean_function}}
                [-off_state {state_name {boolean_function}}]
```

```
Ex:
create_power_switch \
                SW0 \
                -domain PD1 \
                -output_supply_port {VDDM1P2V VDDM1P2V} \
                -input_supply_port  {VDD1P2V VDD1P2V} \
                -control_port {PGEN PGEN} \
                -on_state {full_on VDD1P2V {~PGEN}} \
                -off_state {full_off {PGEN}}
```

Create Isolation Cell

- Set isolation cell from PD1 to PDTOP

```
Syntax:
set_isolation
                isolation_strategy_name
                -domain power_domain
                [-isolation_supply isolation_supply_set]
                [-isolation_signal isolation_signal]
                [-isolation_sense low | high]
                [-clamp_value 0 | 1 | latch]
                [-applies_to inputs | outputs | both]
                [-source source_supply_set_name]
                [-sink sink_supply_set_name]
                [-diff_supply_only true | false]
                [-elements objects]
```

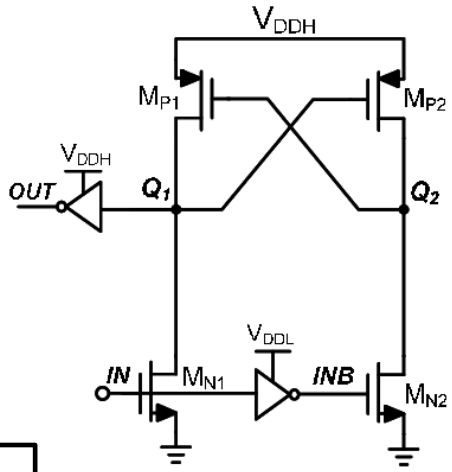



```
Ex:
set_isolation ISOPD1 -domain INST1/PD1 –isolation_supply SS1 \
–clamp_value 0 -applies_to outputs -source SS2 –sink SS0 \
–isolation_signal ISOEN –isolation_sense high \
-elements {INST1/out*}
```

Create Level Shifter

- Create level shifter from PD0 to PD1

```
Syntax:
set_level_shifter
                level_shifter_name
                -domain domain_name
                [-elements list]
                [-exclude_elements list]
                [-applies_to inputs | outputs | both]
                [-source source_supply_set_name]
                [-sink sink_supply_set_name]
                [-input_supply input_supply_set_name]
                [-output_supply output_supply_set_name]
```

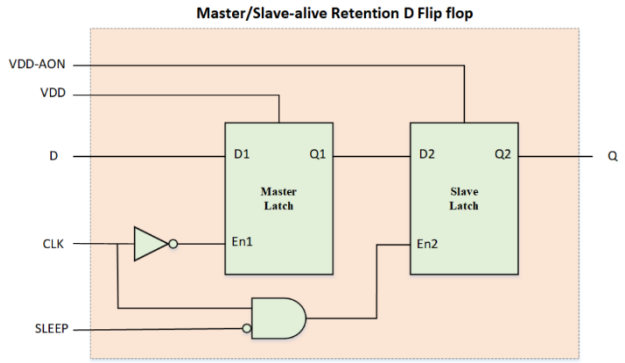



```
Ex:
set_level_shifter LVLPD0 -domain INST0/PD0 -applies_to \
outputs -source SS0 –sink SS1 -elements {INST0/out*}
```

Create Retention Flip Flop

- Create retention flip flop for PD1

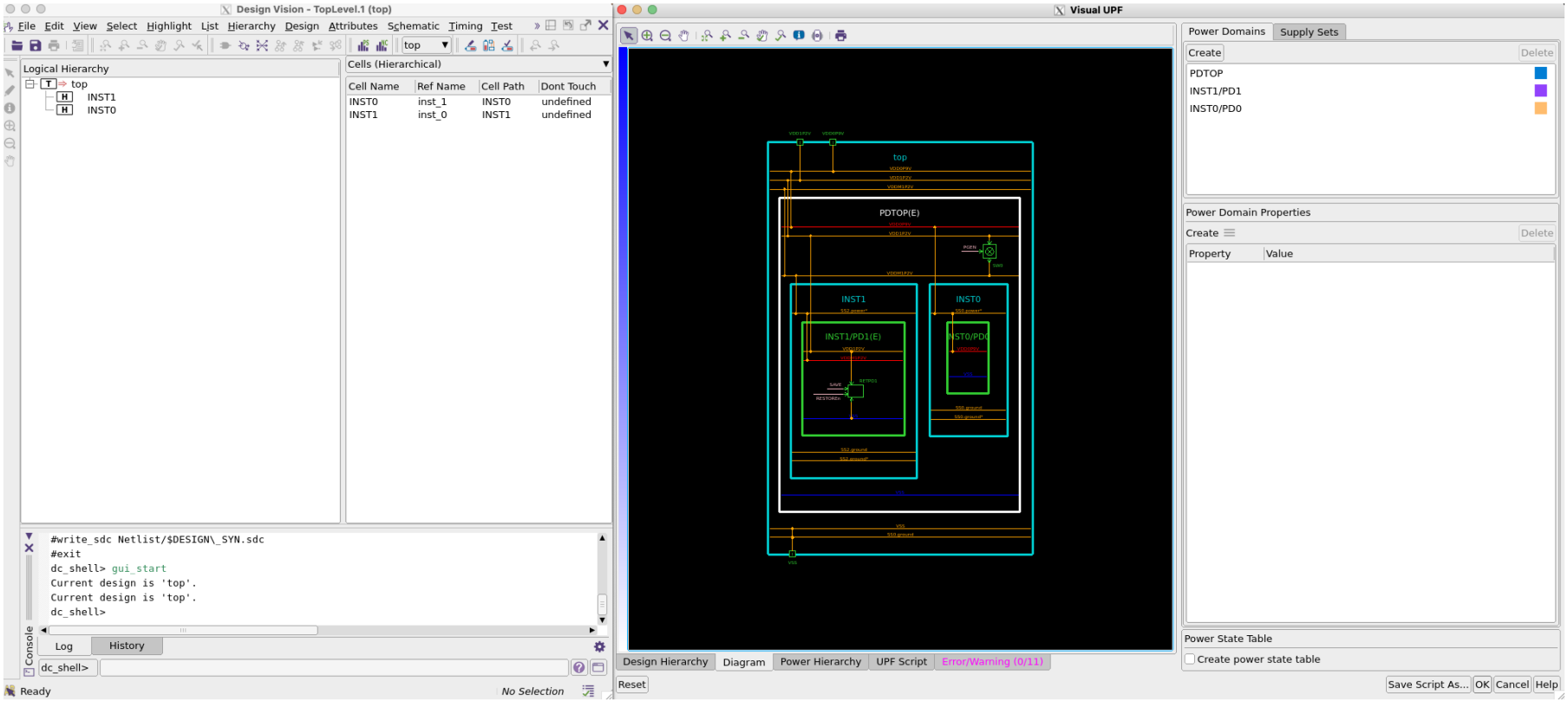
```
Syntax:
set_retention
                retention_strategy
                -domain power_domain
                [-retention_supply retention_supply_set]
                [-elements objects]
                [-save_condition {boolean_function}]
                [-restore_condition {boolean_function}]
                [-retention_condition {boolean_function}]
                [-exclude_elements objects]
                [-save_signal {save_signal save_sense}]
                [-restore_signal {restore_signal restore_sense}]
```




```
Ex:
set_retention RETPD1 –domain INST1/PD1 –retention_supply SS1 \
-elements {INST1/out_reg*} -save_signal {SAVE high} –save_condition {SAVE} \
-restore_signal {RESTOREn low} –restore_condition {!RESTOREn}
```



Design Vision with UPF Diagram





Outline

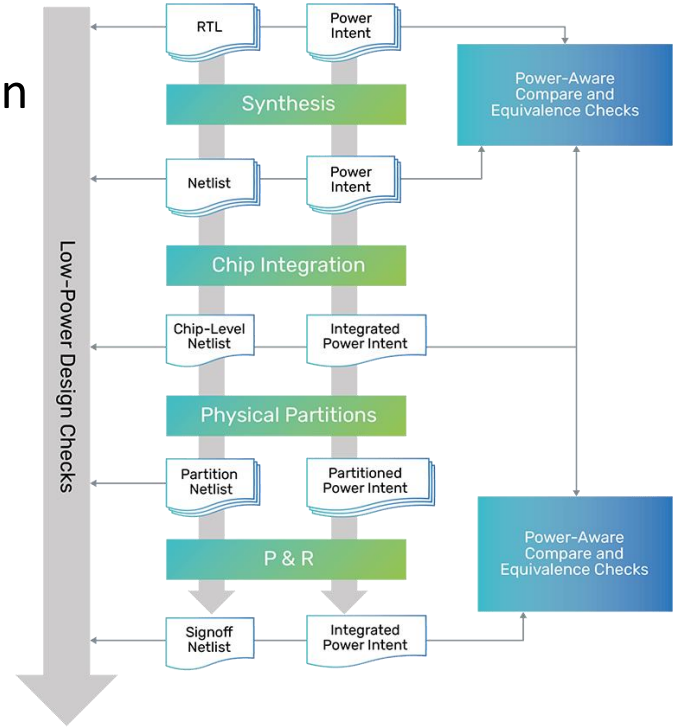
- Review on Previous Logic Synthesis
- Physical Synthesis
- Low-power Design and UPF Flow
- **Conformal Low-power Check**



Conformal Low Power Check

- Conformal is a logic equivalent check (LEC) tool (industrial golden tool)
  - RTL vs gate level netlist
  - Gate level netlist to PnR Netlist

- Conformal low power check (CLP) is the golden for power intent verification
  - Library-consistency checks
  - Power-intent quality checks
  - Structural checks
  - User-defined, rule-based checks
  - Functional checks
  - Power-aware circuit analysis checks





Log and File Setup

- Sets up the log file for the session

```
Syntax:
set log file my_ss.log –replace
```

- Configures the script to continue running even if an error occurs

```
Syntax:
set dofile abort off
```

- Specifies the cell that are not in library as a black box

```
Syntax:
set undefined cell Black_box
```



Low-Power Options

- The set lowpower option commands configure various low-power design settings

```
Syntax:
Set lowpower option
                [-native_1801]
                [-analysis_style | pre_syn | post_syn | post_route]
```

- native_1801 is used to active power intent version of UPF-1801
- analysis_type is used to indicate the design stage for analysis



Set Library and LEF File

- Read liberty file

```
Syntax:
read library -liberty -lp $your_lib_list
```

- Reads in the LEF files, which define the physical layout information

```
Syntax:
read lef file $your_lef_list
```

- Add the search path for files

```
Syntax:
add search path –design $your_search_path
```



Read Design and Run

- Reads the Verilog or other design files, using various flags for handling black boxes and other options.

```
Syntax:
read design –merge bbox –noelab –lastmod –verilog2k –f $your_verilog_file_list
```

- Elaborate the design

```
Syntax:
elaborate design –root $your_root_design
```

- Read the power intent

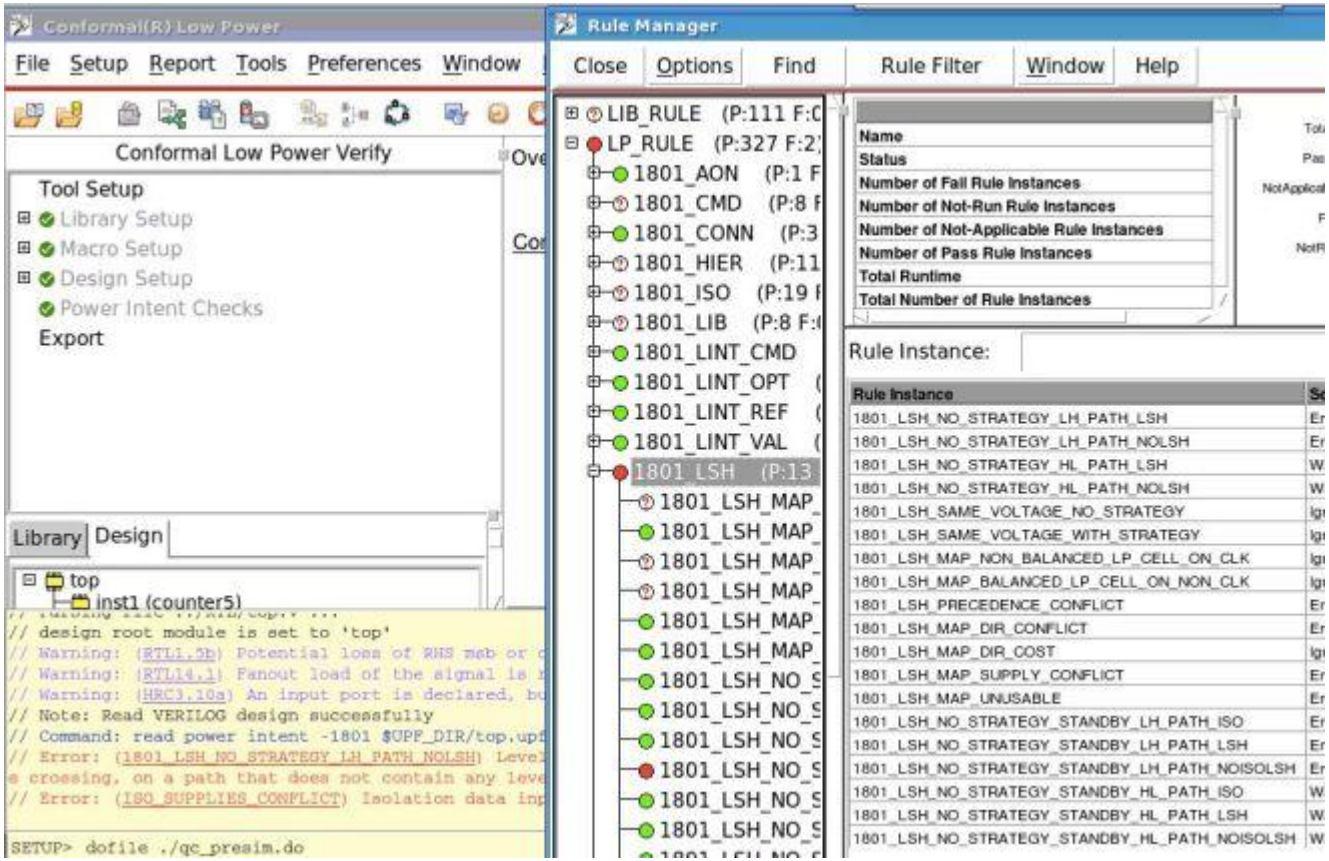
```
Syntax:
Read power intent -1801 $your_upf
```

- Run the script

```
In Terminal run:
lec –lp –verify -1801 -gui –dofile your_dofile.do
```

CLP Report

- Right click the error rule and choose “report rule check” to debug
- Use “report lowpower info” and “report crossing path” to debug





Low Power Simulation with VCS NLP

- VCS NLP
  - VCS Native Low Power (NLP) supports power-aware simulation with UPF/CPF formats, enabling dynamic verification of low-power features.
- Power-Aware Features
  - VCS NLP allows the simulation of various power management functions according to the UPF, such as power-up, power-down, retention, isolation, and level shifting
- Low Power Simulation
  - VCS NLP has built-in low-power simulation capabilities, which previously would have required external tools like MVSIM (a multisupply voltage simulator) for concurrent simulation with VCS.

- Put “$fsdbDumpvars(“+power”) ;” in your tb



How to Run VCS NLP

- Example bash script

```
#!/bin/bash

# Set your file paths
UPF_FILE="path/to/your.upf"
VERILOG_FILES="path/to/your_design.v path/to/another_module.v"
TOP_MODULE="top_module_name"

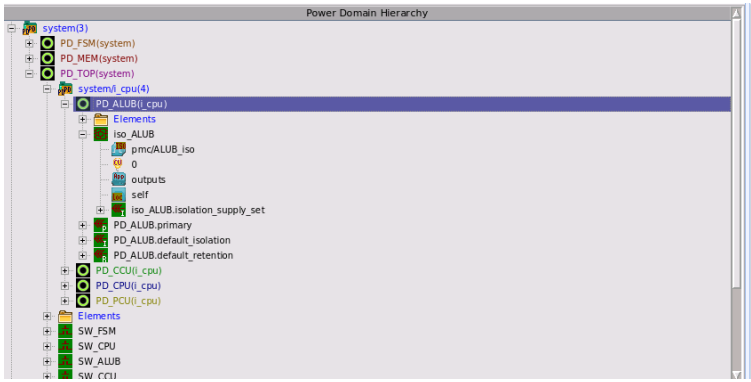
# Compile and run the design
vcs -sverilog -debug_access+all -full64 -power -upf $UPF_FILE \
    $VERILOG_FILES -top $TOP_MODULE -o simv -R
```

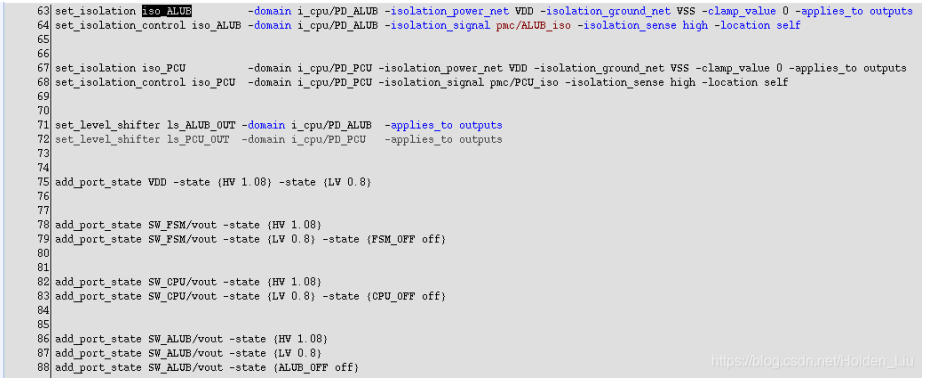


Use Verdi Power Aware Debug

```
In Terminal run:
Verdi –ssf your.fsdb
```

- Different color shows different power domain
- The tool add the virtual isolation cell and the isolation cell strategy is shown as right figure.





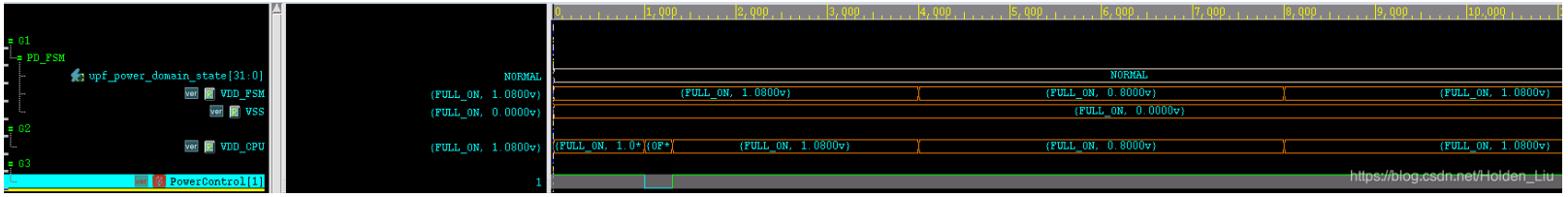
- Click on Power → power state table you can see the power state.

Power State Table

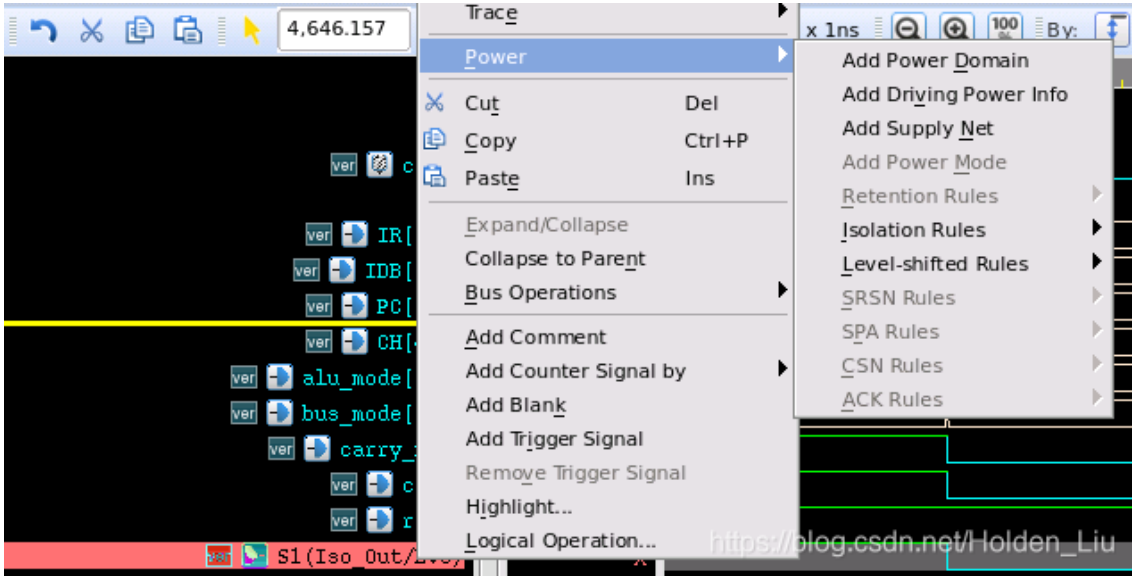
SYSTEM_PST	VDD	SW_FSM/vout	SW_CPU/vout	SW_ALUB/vout	SW_CCU/vout	SW_PCU/vout	SW_MEM/vout
Normal	FULL_ON(1.08v)	FULL_ON(1.08v)	FULL_ON(1.08v)	FULL_ON(1.08v)	FULL_ON(1.08v)	FULL_ON(1.08v)	FULL_ON(1.08v)
_power	FULL_ON(0.8v)	FULL_ON(0.8v)	FULL_ON(0.8v)	FULL_ON(0.8v)	FULL_ON(0.8v)	FULL_ON(0.8v)	FULL_ON(0.8v)
_down	FULL_ON(1.08v)	FULL_ON(1.08v)	OFF(0v)	OFF(0v)	OFF(0v)	OFF(0v)	FULL_ON(1.08v)

Use Verdi to Trace Waveform with UPF

- Add supply net/port and control signal to nWave

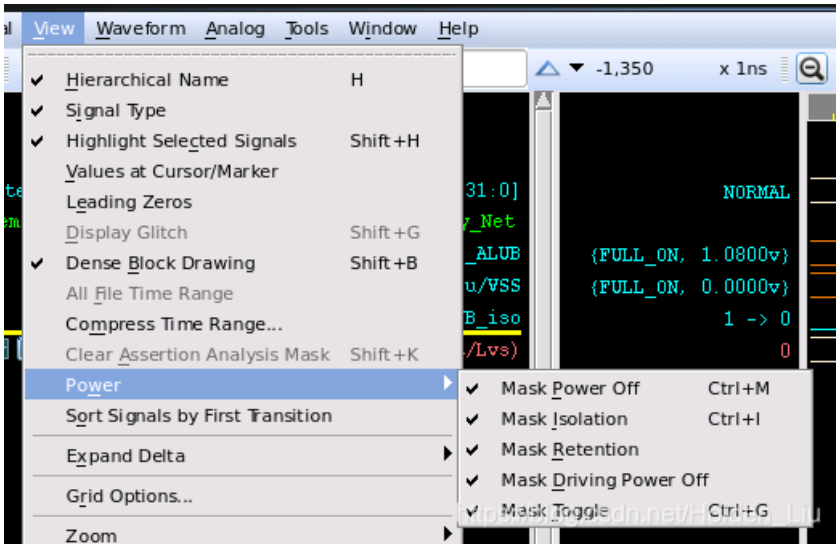


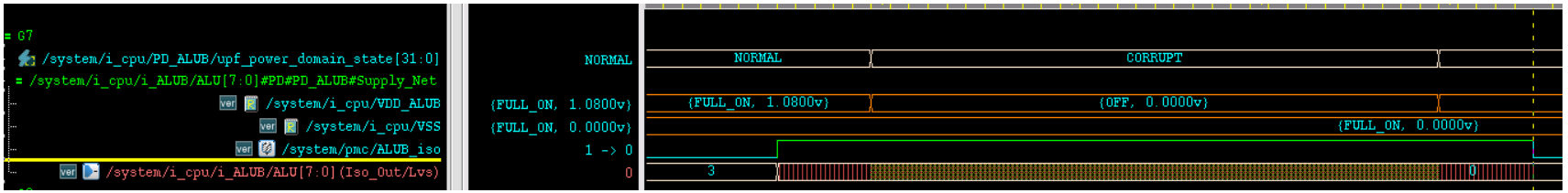
- Select signal → power your can add domain, supply net … information



Waveform Mask and List X

- nWave has five waveform mask to help debugging
- Useful unknown signal tracing tool (Tools →List X)







UPF Case Study: ADFP BoomTile Core - 1

- Boom Core is a RISC-V Core that support RV64GC ISA
- This section shows the settings of design attribute and port attribute.

```
##########################################################################################
#
#  2020.05.29
#
##########################################################################################
set_design_attributes   \
    -elements {.}       \
    -attribute enable_state_propagation_in_add_power_state true \
    -attribute enable_bias true

set_design_attributes   \
    -elements {.}       \
    -attribute lower_domain_boundary false

set_port_attributes     \
    -elements {.}       \
    -attribute related_supply_default_primary true
```

This attribute help power state setting of SS to propagate automatically to lower-level functional nets

This attribute enables body bias

This attribute independently separates lower-level domain contained in higher-level domain.

The tool assumes that the domain's primary supply is the default supply.



UPF Case Study: ADFP BoomTile Core - 2

- Create supply set for CPU Core, and DCACHE.
- SS_VDD is connected to true VDD and VSS
- SS_VVDD is connected to virtual VDD and VSS
- SS_VVDD_DCACHE is connected to Virtual VDD and VSS
- SS_VDD_i is connected to virtual VDD and VSS
- SS_VDDM_i is connected to virtual VDD and VSS

```
################################################################################
#  Create Supply Set
################################################################################
create_supply_set SS_VDD
create_supply_set SS_VVDD
create_supply_set SS_VVDD_DCACHE

create_supply_set SS_VDD_i
create_supply_set SS_VDDM_i
```



UPF Case Study: ADFP BoomTile Core - 3

- This section create power domain of CPU Core (PD_BLK) and DCACHE (PD_DCACHE).
- SS_VVDD and SS_VVDD_DCACHE are attached to PD_BLK and PD_DCACHE.

```
###############################################################################
#  Create Power Domain
###############################################################################
create_power_domain PD_BLK -include_scope
create_power_domain PD_DCACHE -elements {dcache}

###############################################################################
#  Assign Domain Supply
###############################################################################
associate_supply_set SS_VVDD -handle PD_BLK.primary
associate_supply_set SS_VVDD_DCACHE -handle PD_DCACHE.primary
```



UPF Case Study: ADFP BoomTile Core - 3

- This section create the supply ports and connect the supply nets to ports (VDD/VSS).
- Internal supply nets are also created (VVDD, VVDD_DCACHE, VDD_i and VDDM_i).

```
################################################################################
#  Power Supply Network
################################################################################
create_supply_port  VDD
create_supply_net   VDD
connect_supply_net  VDD -ports VDD

create_supply_port  VSS
create_supply_net   VSS
connect_supply_net  VSS -ports VSS

create_supply_net   VVDD -resolve parallel
create_supply_net   VVDD_DCACHE -resolve parallel

create_supply_net VDD_i -resolve parallel
create_supply_net VDDM_i -resolve parallel
```

"Parallel" command allows the supply net to be driven by multiple power switches simultaneously. Multiple outputs can be ON at the same time if their voltage values are the same.



UPF Case Study: ADFP BoomTile Core - 4

- This section set the power switch to PD_BLK.
- The power switch is mapped to P-Type header cell HDR30XSICWD1BWP20P90

```
##########################################################################################
#  Power Gating
##########################################################################################
create_power_switch psw_blk \
    -domain PD_BLK \
    -input_supply_port  {TVDD VDD} \
    -output_supply_port {VDD VVDD} \
    -control_port       {NSLEEPIN PSW_NSLEEPIN_BOOM} \
    -ack_port       {NSLEEPOUT PSW_ACK_BOOM} \
    -on_state           {state_on TVDD {NSLEEPIN}} \
    -off_state          {state_off {!NSLEEPIN}}

map_power_switch psw_blk \
    -domain PD_BLK \
    -lib_cells {HDR30XSICWD1BWP20P90}

set_related_supply_net \
    -power {VDD} \
    -object_list {PSW_NSLEEPIN_BOOM PSW_ACK_BOOM}
```



UPF Case Study: ADFP BoomTile Core - 5

- This section set the power switch to PD_DCACHE.
- The power switch is mapped to P-Type header cell HDR30XSICWD1BWP20P90

```
create_power_switch psw_dcache \
    -domain PD_DCACHE \
    -input_supply_port  {TVDD VDD} \
    -output_supply_port {VDD VVDD_DCACHE} \
    -control_port       {NSLEEPIN PSW_NSLEEPIN_DCACHE} \
    -ack_port       {NSLEEPOUT PSW_ACK_DCACHE} \
    -on_state           {state_on TVDD {NSLEEPIN}} \
    -off_state          {state_off {!NSLEEPIN}}

map_power_switch psw_dcache \
    -domain PD_DCACHE \
    -lib_cells {HDR30XSICWD1BWP20P90}

set_related_supply_net \
    -power {VDD} \
    -object_list {PSW_NSLEEPIN_DCACHE PSW_ACK_DCACHE}

set_related_supply_net \
    -power {VDD} \
    -object_list {SD_CTRL SLP_CTRL DSLP_CTRL}
```



UPF Case Study: ADFP BoomTile Core - 5

- This section updates the supply sets for their power, ground and pmos/nmos body bias.

```
##########################################################
#  Update Supply Set
##########################################################
create_supply_set SS_VDD \
    -function {power VDD} \
    -function {ground VSS} \
    -function {nwell VDD} \
    -function {pwell VSS} \
    -update

create_supply_set SS_VVDD \
    -function {power VVDD} \
    -function {ground VSS} \
    -function {nwell VDD} \
    -function {pwell VSS} \
    -update
```

```
create_supply_set SS_VDD_i \
    -function {power VDD_i} \
    -function {ground VSS} \
    -function {nwell VDD} \
    -function {pwell VSS} \
    -update

create_supply_set SS_VDDM_i \
    -function {power VDDM_i} \
    -function {ground VSS} \
    -function {nwell VDD} \
    -function {pwell VSS} \
    -update
```



UPF Case Study: ADFP BoomTile Core - 5

- This section defines overdrive (OD), Max frequency (Fmax), ground (GND), and off (OFF) states for supply sets.

```
add_power_state SS_VDD \
    -state OD { -supply_expr { power == `{FULL_ON, 0.88} } } \
    -state Fmax { -supply_expr { power == `{FULL_ON, 0.72} } } \
    -state GND { -supply_expr { ground == `{FULL_ON, 0.000} } }

add_power_state SS_VVDD \
    -state OD { -supply_expr { power == `{FULL_ON, 0.88} } } \
    -state Fmax { -supply_expr { power == `{FULL_ON, 0.72} } } \
    -state GND { -supply_expr { ground == `{FULL_ON, 0.000} } } \
    -state OFF { -supply_expr { power  == `{OFF} } }

add_power_state SS_VVDD_DCACHE \
    -state OD { -supply_expr { power == `{FULL_ON, 0.88} } } \
    -state Fmax { -supply_expr { power == `{FULL_ON, 0.72} } } \
    -state GND { -supply_expr { ground == `{FULL_ON, 0.000} } } \
    -state OFF { -supply_expr { power  == `{OFF} } }

add_power_state SS_VDD_i \
    -state OD { -supply_expr { power == `{FULL_ON, 0.88} } } \
    -state Fmax { -supply_expr { power == `{FULL_ON, 0.72} } } \
    -state GND { -supply_expr { ground == `{FULL_ON, 0.000} } }

add_power_state SS_VDDM_i \
    -state OD { -supply_expr { power == `{FULL_ON, 0.88} } } \
    -state Fmax { -supply_expr { power == `{FULL_ON, 0.72} } } \
    -state GND { -supply_expr { ground == `{FULL_ON, 0.000} } }
```



UPF Case Study: ADFP BoomTile Core - 6

- Create power state table for different supply sets

```
###################################################################################
#  Power State Table
###################################################################################

create_pst PST -supplies { \
SS_VDD.power \
SS_VVDD.power \
SS_VVDD_DCACHE.power \
SS_VDD_i.power \
SS_VDDM_i.power \
VSS \
}
add_pst_state OD -pst PST -state {OD OD OD OD OD GND}
add_pst_state Fmax -pst PST -state {Fmax Fmax Fmax Fmax Fmax GND}
add_pst_state DCACHE_OFF -pst PST -state {Fmax Fmax OFF Fmax Fmax GND}
add_pst_state OFF -pst PST -state {Fmax OFF OFF Fmax Fmax GND}
```

State \ Supply	SS_VDD.power	SS_VVDD.power	SS_VDD_DCACHE.power	SS_VDD_i.power	SS_VDDM_i.power	VSS
OD	OD	OD	OD	OD	OD	GND
Fmax	Fmax	Fmax	Fmax	Fmax	Fmax	GND
DCACHE_OFF	Fmax	Fmax	OFF	Fmax	Fmax	GND
OFF	Fmax	OFF	OFF	Fmax	Fmax	GND



UPF Case Study: ADFP BoomTile Core - 7

- Create isolation cell for power domain crossing

```
##########################################################################################
#  ISO
##########################################################################################

# DCACHE => BOOM
set_isolation dcache_iso_cl0            \
    -domain PD_DCACHE                   \
    -isolation_supply SS_VVDD       \
    -clamp_value 0                      \
    -applies_to outputs

set_isolation_control dcache_iso_cl0    \
    -domain PD_DCACHE                   \
    -isolation_signal ISO_EN_DCACHE     \
    -isolation_sense high               \
    -location parent

set_isolation dcache_noiso -domain PD_DCACHE -no_isolation -elements {dcache/PSW_ACK_DCACHE}
```



UPF Case Study: ADFP BoomTile Core - 8

- Connect supply nets to SRAM macros

```
##########################################################################################
#  Connect MACRO Supply net
##########################################################################################
set macroList_1 [find_objects . -object_type model -pattern {TS*} -transitive TRUE -leaf_only]
foreach inst $macroList_1 {
    connect_supply_net VDD -ports $inst/VDD
    connect_supply_net VDD -ports $inst/VDDM
    connect_supply_net VSS -ports $inst/VSS
    connect_supply_net VDD_i -ports $inst/VDD_i
    connect_supply_net VDDM_i -ports $inst/VDDM_i
}
```